

Laboratorio de Lenguaje Emergente

Informe y conclusiones

Entrega del domingo 27 de septiembre de 2026

Este documento reúne, en un solo lugar, las conclusiones con las que cierra cada notebook de la entrega (00_dataset a 05_judge), el cierre del proyecto, el registro de decisiones y el uso de IA. El código, las salidas completas y todas las muestras generadas están en los notebooks ejecutados: corrieron de punta a punta en una notebook con RTX 5070 Laptop (8 GB), con semilla 1337.

El hilo que atraviesa todo, y que defendemos en el cierre del proyecto: en nuestros modelos la *forma* aparece antes que el *contenido*. Pasa en el preentrenamiento (la gramática llega antes que la consistencia), en el SFT (el formato se instala en unos cientos de pasos; usar las palabras pedidas, a medias) y en la evaluación (la perplejidad mide cuánto se parece un texto al corpus, no si es un buen cuento).

Contenido

1. Etapa 0 · El dataset como instrumento
2. Etapa 1 · Tokenización: un BPE propio
3. Etapa 2 · Preentrenamiento y ablación ancho contra profundo
4. Etapa 3 · La representación aprendida
5. Etapa 4 · SFT: comportamiento, no conocimiento
6. Etapa 5 · Medir la emergencia con un juez local
7. Cierre del proyecto: qué muestra el pipeline completo
8. Registro de decisiones del proyecto
9. Uso de IA
10. Anexo · Etapa 1: cómo medimos lo que no imprimen las celdas

Etapla 0 · El dataset como instrumento

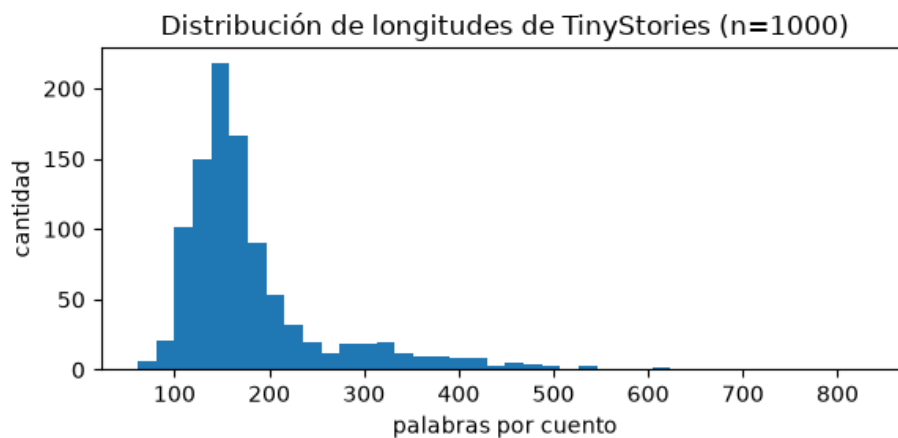


Figura 1. Largo de los cuentos, en palabras (1.000 cuentos del split `train`).

Lectura de lo que medimos

Esta inspección corrió con el valor por defecto del notebook (`SMOKE_TEST = True`: 1.000 cuentos del split `train` y 200 registros de `Instruct`). Alcanza para ver el formato y el orden de magnitud; las medidas finas, en tokens, están en la Etapa 1, sobre 5.000 cuentos held-out y 100.000 de entrenamiento.

- **Largo:** 183,8 palabras por cuento en promedio, con el grueso entre 100 y 250 y una cola larga a la derecha (mínimo 61, máximo 837). En tokens de nuestro BPE (Etapa 1, sobre 5.000 cuentos held-out del split `validation`): 196 por cuento en promedio, mediana 181, y con `block_size = 256` entra entero el 90 %. **Un detalle que se nos había pasado:** los cuentos de `train` son más largos que los de `validation` (183,8 palabras contra 158,5 en nuestras muestras; 221 tokens por cuento contra 201 en los datos de la Etapa 2, contando el `<|endoftext|>`), así que ese 90 % es optimista para los datos con los que efectivamente entrenamos.
- **Vocabulario:** unas 4.961 palabras distintas en ~184.000 palabras de texto, y las diez más comunes (*the, and, to, a, was, she, he, they, it, her*) son, solas, el 28,5 % de todo lo que se lee. Son palabras de función de un relato en pasado y en tercera persona: *was* aparece más que cualquier sustantivo.
- **Instruct:** el registro de ejemplo trae `Features: Dialogue, Words: quit, oak, gloomy` y un `Summary:` antes de `Story:`, y el cuento usa de verdad las tres palabras pedidas (*oak*, tres veces). Esa asociación instrucción → cuento es la que la Etapa 4 intenta enseñar.

Preguntas para el informe

¿Qué les da experimentalmente "el vocabulario de un chico de 3–4 años" que un corpus general de texto web no les da? En los 1.000 cuentos que miramos encontramos unas 4.961 palabras distintas y un promedio de 183,8 palabras por cuento. Como el vocabulario es chico y los cuentos tienen un estilo parecido, el modelo ve muchas veces las mismas palabras y formas de escribir. Con pocos parámetros y la misma cantidad de datos, eso le facilita aprender a armar cuentos que se entiendan. Nos sirve para estudiar qué puede aprender con datos simples y repetidos. Igual, que escriba bien estos cuentos no significa que entienda cualquier tema.

Si entrenaran la misma arquitectura sobre un pedazo de texto crudo de internet con la misma cantidad de tokens, ¿qué esperarían que cambie, y por qué? Si usamos texto web, el modelo vería más temas, palabras poco comunes, nombres, enlaces y hasta restos de HTML. Entonces tendría menos ejemplos repetidos de cada forma de escribir. Además, si dejamos el mismo tokenizador, algunas palabras se separarían en más tokens y ocuparían más espacio del contexto. Esperaríamos que le cueste más escribir cuentos cortos y coherentes como los de TinyStories, aunque esto habría que probarlo. Para compararlos bien, usaríamos el mismo tokenizador y los mismos textos de prueba; no alcanza con mirar la pérdida de cada modelo en su propio corpus porque son textos diferentes.

Ampliación: el mismo argumento, con los números de todas las etapas

Las respuestas de arriba, apoyadas en lo que medimos en las Etapas 1 a 5, y la pregunta con la que la consigna abre esta etapa: por qué un vocabulario chico a propósito convierte al dataset en un instrumento de laboratorio.

Por qué un vocabulario chico a propósito es un instrumento de laboratorio

Porque achica el problema hasta que un modelo de nuestro tamaño deja de estar desbordado, y entonces lo que aprende —o no aprende— se puede atribuir a los datos y no a que "le faltó escala".

Un experimento sirve cuando se mueve una variable y el resto queda quieto. Con texto general, un modelo de 6M de parámetros fallaría por todo a la vez: demasiadas palabras, temas, registros y datos del mundo para su tamaño. Ese fracaso no diría nada, porque no habría forma de separar el efecto de los datos del de la escala. TinyStories reduce lo que hay que aprender (léxico, sintaxis y un mundo de pocos personajes y objetos) a algo que entra en el modelo, y recién ahí se puede preguntar qué aprende primero, qué necesita más capas o qué cambia con el SFT. Es la lógica de los organismos modelo en biología: la mosca de la fruta no es "un animal cualquiera", es uno lo bastante simple como para experimentar y lo bastante completo como para que el fenómeno aparezca.

Nuestros números lo muestran: con 5,85M de parámetros y 6 minutos de GPU, el modelo ancho pasa de adivinar al azar (pérdida 9,06 $\approx \ln 8.192$) a sus primeras oraciones bien formadas en unos 500 pasos (pérdida 3,3), y termina en perplejidad 10,1 (Etapa 2).

Qué nos da el vocabulario chico, con números

1. **Repetición.** Pocas palabras se repiten muchísimo: diez palabras son el 28,5 % del texto y, con nuestro BPE, los 1.000 tokens más usados cubren el 90 % (Etapa 1). Casi todas las filas de la tabla de embeddings reciben miles de ejemplos, y por eso en la Etapa 3 los vecinos de *dog* o de *sad* terminan teniendo sentido.
2. **Un tokenizador que es casi un diccionario.** Con 8.192 entradas, solo el 0,9 % de las palabras se parte en más de un token (Etapa 1): los tokens son palabras, se pueden analizar como palabras (Etapa 3), y un cuento entero entra en la ventana de 256.
3. **Un solo género.** Narración en pasado, en tercera persona, con diálogo y fórmulas fijas: el 67 % de los cuentos held-out empieza con *Once upon a time* (Etapa 1). Con una estructura global tan regular, la coherencia pasa a ser algo que un modelo chico puede llegar a aprender, y algo que se puede medir (Etapas 2 y 5).
4. **Un mundo chico.** Pocos personajes y objetos, causas simples. Eso permite preguntar por consistencia (¿el personaje sigue siendo el mismo? ¿usa las palabras que le pedimos?) sin que la respuesta dependa de conocimiento enciclopédico que un modelo de 6M no puede guardar.
5. **Experimentos baratos.** El pipeline entero corre en poco más de media hora en la GPU de una notebook. Eso es lo que nos dejó hacer la ablación, el SFT, LoRA y el juez, en lugar de elegir uno.

Lo que no le creemos del todo

"El vocabulario de un chico de 3–4 años" es una intención del generador, no una restricción dura. El paper armó los cuentos pidiéndole a GPT-3.5/4 que combinara palabras de una lista de unas 1.500 básicas, pero en solo 1.000 cuentos aparecen unas 5.000 formas distintas (con flexiones y nombres propios), y en nuestro vocabulario BPE entraron *accomplishment*, *determination* y *compassionate* (Etapa 1). El corpus trae además impurezas de su origen sintético: el 6 % de los cuentos tiene texto mal codificado (â€™ en lugar de '), que el modelo aprende como cualquier otro patrón. Y la misma regularidad que lo hace útil baja la vara: "coherente" acá quiere decir coherente para un cuento infantil de fórmula, y parte del éxito de un modelo chico es haber aprendido la fórmula.

La predicción sobre texto web, con lo que medimos

Texto con la pinta de internet pero sin hilo: fluidez local, poca coherencia, y un modelo que gasta casi todo su tamaño en cubrir palabras que ve pocas veces. No lo corrimos; es una predicción, apoyada en lo que sí medimos:

- **El mismo presupuesto de tokens compraría menos texto.** Un BPE de 8.192 entradas entrenado sobre la web tiene que repartirse entre muchas más palabras, nombres, números, código y otros idiomas, así que parte más palabras: más tokens por palabra, menos palabras en los mismos 82M tokens y menos documento en cada ventana de 256. El tokenizador de GPT-2, entrenado sobre texto web, ya muestra ese reparto: entre sus primeros merges están los de la prosa informativa (*ion*, *ic*, *of*), que en el nuestro aparecen cientos de merges después (Etapa 1, anexo B).
- **Cada palabra se vería muchas menos veces.** En la web la cola larga es mucho más pesada: muchas filas de la tabla de embeddings quedarían casi sin entrenar (la trampa de *SolidGoldMagikarp* que discutimos en la Etapa 1), y la tabla de embeddings más la capa de salida ya son el 73 % de nuestro modelo ancho.
- **Mezcla de registros.** Noticias, listas, foros, código: el modelo repartiría su capacidad en imitar formatos distintos. Esperaríamos algo parecido a lo que vimos en clase con el Transformer de caracteres sobre Shakespeare: la pinta visual del texto, sin sentido.
- **La pérdida no se podría comparar directo.** Como decimos arriba, habría que medir a los dos modelos sobre los mismos textos de prueba; y si cambia el tokenizador, la pérdida por token cambia de escala y hay que pasarla a bits por carácter (Etapa 1, respuesta 3).
- **Y el laboratorio dejaría de funcionar.** Si el ancho y el profundo fallan los dos, no hay forma de saber si el problema es la forma del modelo o que no le alcanza el tamaño: ese es el "ruido de la escala" que TinyStories saca del medio. El paper lo encontró en grande: modelos de ~125M de parámetros entrenados sobre corpus generales casi nunca llegan a inglés coherente, y modelos de menos de 10M entrenados sobre TinyStories sí.

El experimento que lo contestaría es entrenar el mismo modelo ancho con 82M tokens de una muestra web (y un BPE de 8.192 entrenado sobre esa muestra) y comparar bits por carácter, muestras y notas del juez. No lo hicimos: es otra corrida entera de las Etapas 1 y 2, y quedó afuera por tiempo (ver el registro de decisiones, abajo).

Etapas 1 · Tokenización: un BPE propio

Qué encontramos, en corto

- **El tokenizador es un resumen del corpus.** Sus primeros merges arman *the, and, to, he, was* y el sufijo *-ed*: el esqueleto de un cuento contado en pasado.
- **Con un vocabulario 6 veces más chico que el de GPT-2, partimos los mismos cuentos en un 1,7% menos de tokens.** En un corpus de vocabulario angosto, con 8.192 tokens casi cada palabra ya es un token, y agrandar `V` no compra casi nada.
- **El vocabulario se paga en parámetros:** la tabla de embeddings y la capa de salida suman 4,19M, cerca del 72% del modelo ancho de la Etapa 2.
- **Achicarlo tampoco es gratis:** con `V = 512` los cuentos quedan un 78% más largos, y un tercio de ese vocabulario son bytes que no aparecen nunca.
- **Con `block_size = 256` entra entero el 90% de los cuentos,** aunque que un cuento "entre" no quiere decir que el modelo lo vea entero.

Abajo siguen la lectura de cada resultado, las respuestas a las preguntas, las decisiones y las dudas que nos quedan; el anexo, al final de este informe, muestra cómo medimos los números que no imprimen las celdas.

Lecturas de cada resultado

Lectura: qué le pasa a un cuento real

- **Ida y vuelta: 5.000 de 5.000.** Como el alfabeto de partida son los 256 bytes, cualquier texto se puede escribir con estos tokens y no se pierde nada. Ojo: que no pierda información no dice si corta *bien* el texto. Eso lo miden las celdas que siguen.
- **Las palabras no se parten.** Los primeros 40 tokens del cuento de ejemplo del notebook son 29 palabras y 11 signos (puntuación y comillas). Ninguna palabra quedó cortada, ni siquiera `Ġpolish` o `Ġreplied`. En este corpus el BPE se porta casi como un tokenizador de palabras: solo el 0,9% de las palabras termina en más de un token (anexo A).
- **La misma palabra puede ser dos tokens.** El cuento empieza con `Spot` y más adelante aparece `ĠSpot`: son dos filas distintas de la tabla de embeddings. `Ġ` es el espacio, que va pegado a la palabra que sigue, y la primera palabra de un cuento no tiene espacio adelante.
- **Hay mucho diálogo.** `!"` y `."` son un solo token cada uno: la comilla de cierre aparece tan seguido pegada al signo que el BPE los unió temprano, en los merges 177 y 165 (anexo B).

Lectura: qué aprendió primero

Un *merge* es unir dos pedazos que aparecen mucho juntos en un pedazo nuevo. Los 25 primeros se agrupan en tres familias:

- **Comienzos de palabra** (`Ġ` + letra): `Ġt`, `Ġa`, `Ġs`, `Ġw`, `Ġb`, `Ġh`... En prosa simple, lo más frecuente es que empiece una palabra.
- **Pedazos de las palabras más usadas**, que se completan enseguida: `Ġthe` (merge 7), `Ġand` (9), `Ġto` (10), `Ġhe` (23) y `Ġwas` (24).
- **El sufijo *ed*** (merge 8): el pasado de *played, smiled, wanted*.

Con 25 merges ya están *the, and, to, he, was* y *-ed*: el esqueleto de una oración narrada en pasado. Por qué *este* corpus da estos merges lo respondemos al final, comparando contra GPT-2.

El 73% de "palabras enteras" exagera. El filtro (`Ġ` + solo letras) también cuenta comienzos de palabra, como `Ġwa` (el paso previo a `Ġwas`, del merge 14) o `Ġpl`, y pasos intermedios que no se usan nunca solos, como `Ġbutterf`, que solo existe como escalón hacia `Ġbutterfly`. Si contamos solo los tokens que, cuando aparecen, son casi siempre una palabra

completa, quedan 4.917: **el 60% del vocabulario** (anexo G). Sigue siendo casi un diccionario, pero más chico de lo que parece.

Los tokens más largos son palabras largas y bastante abstractas para "el vocabulario de un chico de 3 o 4 años": `Ġaccomplishment`, `Ġdetermination`, `Ġcompassionate`, `Ġencouragement`, `Ġgranddaughter`, y hasta `Ġneighbourhood`, con ortografía británica. Nuestra sospecha, que no chequeamos, es que vienen de las moralejas con que cierran varios cuentos y de que los cuentos los escribió GPT-3.5/4, no un chico. También aparece la misma palabra dos veces por la mayúscula: `ĠUnfortunately` y `Ġunfortunately` son filas distintas.

Un detalle de reproducibilidad. Los últimos lugares de esa lista son empates (hay varios tokens de 13 caracteres) y `get_vocab()` no devuelve un orden fijo, así que pueden cambiar de una corrida a otra. En otra corrida de este mismo notebook salieron `Ġconsequences` y `Ġfirefighters` en lugar de `Ġgrandparents` y `Ġappreciation`. El tokenizador es el mismo; lo único que cambia es cómo se desempata al imprimir.

Lectura: la tabla contra GPT-2

- **Nuestro BPE** usa 195,75 tokens por cuento en promedio (mediana 181), 1,23 tokens por palabra y 4,15 caracteres por token.
- **GPT-2** tiene un vocabulario 6 veces más grande (50.257) y aun así necesita un **1,7% más** de tokens (199,05). **cl100k**, 12 veces más grande, necesita apenas un **2,5% menos** (190,92).
- **El otro lado de la balanza son los parámetros:** solo la tabla de embeddings y la capa de salida suman 4,19M con nuestro vocabulario, 25,73M con el de GPT-2 y 51,34M con cl100k. Con GPT-2 pagaríamos 6 veces más en la parte del modelo que ya es la más pesada, a cambio de secuencias *más largas*.
- **Un ejemplo de por qué GPT-2 pierde:** `Kitty` al principio de un cuento, o pegado a una comilla, es un token para nosotros y dos para GPT-2, `K + itty` (anexo B). Sus 50.257 entradas se repartieron sobre texto de internet de todo tipo; las nuestras, solo sobre cuentos.

El porqué de esta tabla, y qué dice del toma y daca $V \leftrightarrow T$, está en la respuesta 2.

$V \leftrightarrow T$ sobre TinyStories ($n_{\text{embd}} = 256$ para el costo en parámetros)

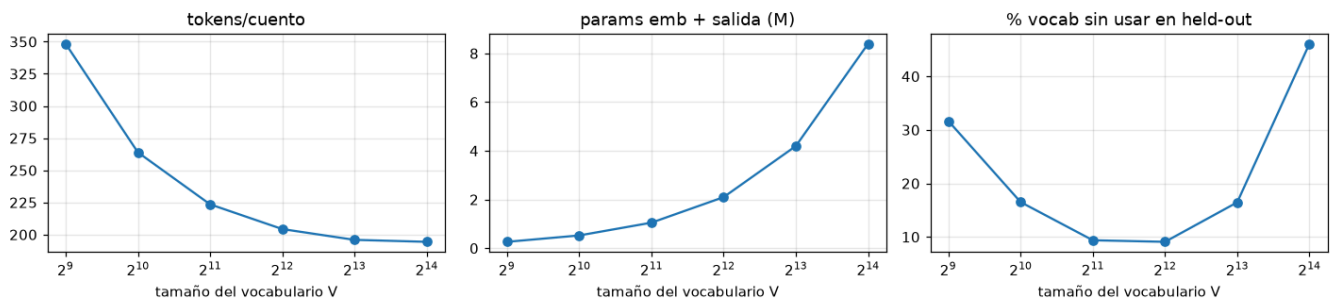


Figura 2. Barrido del tamaño del vocabulario V : tokens por cuento, parámetros de embeddings y salida (con $n_{\text{embd}} = 256$) y porcentaje del vocabulario que no aparece en el held-out.

Lectura: las tres curvas, juntas

Tokens por cuento (izquierda). Cada vez que duplicamos V , ahorramos más o menos la mitad que la vez anterior: -24%, -15%, -8,6%, -4,1% y, de 8.192 a 16.384, apenas -0,8%. La curva se aplana contra un piso. El *pre-tokenizador* (el paso que corta el texto en palabras y signos antes del BPE) no deja que ningún merge junte una palabra con la coma que le sigue, así que ningún vocabulario puede bajar de 193,6 tokens por cuento (anexo A). Con 16.384 ya estamos a un 0,4% de ese piso.

Parámetros (centro). Crecen en línea recta con V : duplicar el vocabulario duplica el costo. La curva parece una exponencial solo porque el eje x está en escala logarítmica; es una trampa visual del gráfico.

Vocabulario sin usar (derecha). La forma de U engaña. Si pasamos los porcentajes a cantidad de filas, el número **sube siempre**: 162, 169, 192, 372, 1.348 y 7.547. Detrás hay dos fenómenos distintos (anexo C):

- **A la izquierda**, casi todo son **bytes que no aparecen nunca** en estos cuentos: caracteres de control, bytes de letras de otros idiomas y símbolos de código como `( ) { } [ ] = _ #`, que no aparecen ni una vez en los 5.000 cuentos. Son unas 160 filas fijas: con `V = 512` son un tercio del vocabulario, y con 8.192, un 2%.
- **A la derecha**, son **merges aprendidos que no aparecen en el held-out**. Algunos son pasos intermedios que un merge posterior se "comió", como `Ġbutterf`. Otros son palabras raras que el tokenizador vio en sus 20.000 cuentos de entrenamiento y que no vuelven a salir. Con 8.192 son 170 bytes + 401 intermedios + 776 raros + `<|endoftext|>`.

Las tres juntas: el codo está entre 2.048 y 8.192. Más abajo, los cuentos se alargan mucho. Más arriba, pagamos el doble de parámetros y 5,6 veces más filas sin usar para ahorrar un 0,8% de tokens.

Dos matices que nos parecen importantes:

- **"Sin usar en 5.000 cuentos" no es lo mismo que "sin entrenar"**. El modelo se entrena con muchos más cuentos, y ahí varias de estas filas aparecen, aunque sea pocas veces. Lo que sí es cierto es que el vocabulario tiene una cola larga: los 1.000 tokens más usados cubren el 90% del texto (anexo C), y las otras ~7.200 filas se reparten el 10% restante, así que se entrenan mucho menos.
- **Una fila que nunca entra igual recibe gradiente a la salida**. Si la capa de salida es una matriz aparte, como supone la cuenta de parámetros, en cada paso el softmax empuja hacia abajo la probabilidad de todos los tokens que no tocaban. Lo que se queda sin entrenar es su fila del *embedding de entrada*. Esa es la trampa que el notebook anticipa para la Etapa 3.

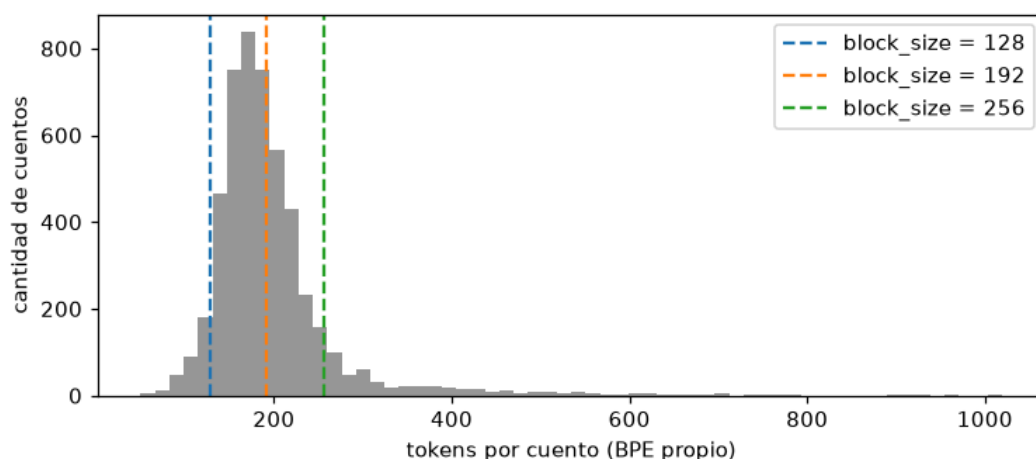


Figura 3. Largo de los 5.000 cuentos held-out en tokens de nuestro BPE, con los tres `block_size` candidatos.

Lectura: cuánto de un cuento entra en la ventana

- **La forma:** una campana alrededor de 180 tokens (mediana 182), con una cola larga a la derecha (p99 = 544; el más largo tiene 1.019). 8 de cada 10 cuentos tienen entre 129 y 256 tokens.
- **Las tres líneas:** con 128 entra entero apenas el 6,1% de los cuentos (la línea cae en la ladera izquierda de la campana); con 192, el 60,4% (queda apenas por encima de la mediana); con 256, el 90,2% (coincide con el p90, que es 255). Para llegar al 99% haría falta una ventana de ~544, más del doble, y el costo de la atención crece con el cuadrado del largo. **Por eso se eligió 256.**
- **Es el mismo codo que en el barrido de V:** cada paso compra menos. De 128 a 192 se suman 54 puntos; de 192 a 256, 30; de 256 a 544, 9.
- **V y block_size son la misma decisión vista de dos lados.** Lo que importa es cuánto *cuento* entra en la ventana, y eso depende de los dos. Con `V = 512` un cuento promedio tiene 348 tokens: una ventana de 256 no alcanzaría ni para el promedio.

Una duda sobre nuestra propia métrica. "Entra entero" mide si un cuento *cabe*, no si el modelo lo *ve* entero. Si el entrenamiento arma cada ventana cortando en una posición al azar el corpus concatenado (todos los cuentos pegados,

separados por `<|endoftext|>`), la ventana casi nunca empieza justo al principio de un cuento. Hicimos la cuenta: con 256, solo ~36% de las ventanas contiene algún cuento completo (~10% con 192), y el 18% de los tokens es de cuentos que no entran en ninguna ventana (anexo D). O sea que el modelo aprende sobre todo con pedazos de cuentos, y tiene que usar `<|endoftext|>` para darse cuenta de que dentro de la misma ventana empezó otra historia.

Respuestas a las preguntas

1. ¿Qué merges aparecieron primero, qué dicen del corpus y por qué no se parecen a los de un corpus de código?

Los primeros merges son los pedazos de un cuento narrado en pasado: comienzos de palabra, *the*, *and*, *to*, *he*, *was* y el sufijo *-ed*. Salen esos porque el BPE es, literalmente, contar: una primero el par de símbolos que más se repite en el corpus, y en estos cuentos lo que más se repite es eso.

El BPE no "entiende" nada: cuenta qué par de símbolos aparece más veces pegado, lo une y repite. Por eso la lista de merges funciona como una huella digital del corpus; mirándola se podría adivinar de qué tipo de texto salió. En TinyStories lo que más se repite es la estructura de un cuento infantil: narración en pasado (*was*, *-ed*), en tercera persona (*he*, *she*), con diálogo (*said*, comillas pegadas a la puntuación) y con un elenco chico de personajes que vuelve una y otra vez: `ĠLily` se vuelve un token en el merge 108, antes que `Ġhappy` (146) o `Ġfriend` (158).

Para comprobar que es el corpus y no el algoritmo, comparamos con GPT-2, que usa el mismo BPE a nivel byte pero entrenado sobre texto de internet. El número es el merge en el que nace cada token (anexo B):

token	nuestro BPE	GPT-2	qué representa
<code>Ġthe</code>	7	7	la palabra más común del inglés: igual en los dos
<code>ed</code>	8	21	pasado
<code>Ġhe</code>	23	84	tercera persona
<code>Ġwas</code>	24	118	narración en pasado
<code>Ġsaid</code>	72	276	diálogo
<code>Ġshe</code>	79	418	tercera persona
<code>Ġof</code>	88	31	prosa informativa (<i>one of the</i> , <i>the number of</i>)
<code>ic</code>	214	36	vocabulario técnico (<i>public</i> , <i>specific</i>)
<code>ion</code>	503	40	vocabulario abstracto (<i>information</i> , <i>nation</i>)

Lo narrativo aparece mucho antes en el nuestro; lo informativo y formal (*of*, *-ion*, *-ic*), mucho antes en GPT-2.

¿Y un corpus de código? No lo probamos, así que es una predicción: los primeros merges serían los espacios de la indentación, símbolos como `( )`, `:` o `=`, y palabras clave como `def`, `return` o `self`. En nuestros datos hay un indicio de qué tan distinto sería: los símbolos típicos del código `( ( ) { } [ ] = _ < > # )` no aparecen **ni una vez** en los 5.000 cuentos held-out (anexo C). Para nuestro tokenizador son filas muertas; en un corpus de código estarían entre los primeros merges.

2. Con el vocabulario elegido, ¿cuántos tokens por cuento da, cómo se compara con GPT-2 y qué dice eso del toma y daca $V \leftrightarrow T$?

195,75 tokens por cuento en promedio (mediana 181), contra 199,05 de GPT-2: con un vocabulario 6 veces más chico, partimos los mismos cuentos en un 1,7% menos de tokens. En un corpus de vocabulario angosto, pasados unos pocos miles de tokens, agrandar V casi no acorta las secuencias y sí agranda mucho el modelo.

El toma y daca $V \leftrightarrow T$ dice que un vocabulario más grande (V) da secuencias más cortas (T), porque cada token cubre más texto. Acá casi no se cumple, porque el corpus "se agota": TinyStories tiene pocas palabras distintas, y con 8.192 tokens casi todas ya son un token entero (solo el 0,9% de las palabras se parte; anexo A). Lo que queda sin comprimir no son palabras sino puntuación, comillas y saltos de línea: el 17% de los tokens no tiene ni una letra. Esos signos el pre-

tokenizador los separa de las palabras *antes* del BPE, así que ningún vocabulario, por grande que sea, puede unir *said* con la coma que le sigue. Por eso hay un piso: con este pre-tokenizador ningún vocabulario puede bajar de **193,6 tokens por cuento**, y nosotros estamos a un 1,1% de él.

Eso explica a los dos tokenizadores grandes:

- **GPT-2 pierde** porque sus 50.257 entradas se gastaron en internet: código, nombres propios, otros idiomas. Para nuestros cuentos le sobran casi todas, y le faltan algunas que nosotros sí tenemos (*Kitty* en un solo token).
- **cl100k gana, pero no por su vocabulario enorme**, sino sobre todo por otras reglas de corte: pega los saltos de línea a la puntuación (un `.` seguido de dos saltos de línea es un solo token). Nosotros gastamos 7,2 tokens por cuento solo en saltos de línea y cl100k 3,7 (anexo A). Solo esa diferencia son 3,5 tokens por cuento, de una ventaja total de 4,8 (195,75 – 190,92).

Nuestra conclusión: en un corpus angosto, el largo de las secuencias depende más de *cómo se corta* el texto que del tamaño del vocabulario. El costo del vocabulario, en cambio, crece sin techo: 4,19M de parámetros para nosotros, 25,73M con GPT-2 y 51,34M con cl100k.

3. ¿Un vocabulario más chico es siempre mejor acá, o hay algo que se paga a cambio?

No es siempre mejor. Un vocabulario chico ahorra parámetros, pero alarga los cuentos, y eso se paga en ventana, en datos vistos y en trabajo para el modelo. Uno grande tampoco sirve: pasado el codo, se paga el doble para no ganar nada. Para nosotros la zona buena va de 2.048 a 8.192, y no estamos seguros de que 8.192 le gane a 4.096.

Lo que se paga con $V = 512$, mirando las tres curvas juntas:

- **Secuencias un 78% más largas** (348 tokens por cuento contra 196). Con una ventana de 256 no entraría ni un cuento promedio.
- **Menos cuentos por el mismo cómputo.** Con un presupuesto fijo de tokens de entrenamiento, el modelo vería un 44% menos de cuentos, porque cada uno gasta 349 tokens en lugar de 197 (contando el `<|endoftext|>`).
- **Más trabajo para el modelo.** Si las palabras llegan partidas, es de esperar que parte de su capacidad se vaya en rearmarlas antes de poder ocuparse del cuento.
- **Y ni siquiera es eficiente en filas:** un tercio de su vocabulario son bytes que no aparecen nunca.

Del otro lado, pasar de 8.192 a 16.384 duplica el costo (de 4,19M a 8,39M) para ahorrar un 0,8% de tokens, y casi la mitad del vocabulario queda sin aparecer en el held-out.

La duda que nos queda es entre 4.096 y 8.192. Con 4.096 los cuentos serían un 4,3% más largos (204 tokens contra 196), pero la tabla de embeddings y la de salida costarían la mitad (2,10M contra 4,19M) y habría 3,6 veces menos filas sin usar en el held-out (372 contra 1.348). A favor de 8.192: es el techo del rango que sugiere la consigna, está cerca de los ~10K que usó el paper de TinyStories, y deja más palabras enteras para analizar en la Etapa 3. El experimento que zanjaría la discusión es preentrenar el mismo modelo con los dos vocabularios y comparar. No lo hicimos, porque implicaba repetir toda la Etapa 2.

Una trampa para quien haga esa comparación: la pérdida por token no se puede comparar entre vocabularios distintos. Con $V = 512$ la pérdida arrancararía en $\ln 512 \approx 6,2$ y no en $\ln 8.192 \approx 9,0$, sin que el modelo sea mejor: cada token "dice" menos. Hay que pasarla a una unidad que no dependa del tokenizador, como bits por carácter: $L / (4,15 \cdot \ln 2)$ para el nuestro, donde L es la pérdida por token en nats y 4,15 son los caracteres por token de la tabla.

Decisiones, diseño y reflexión

Qué elegimos en esta etapa, por qué, y qué nos costó:

1. **BPE a nivel byte, con tokenizers de HuggingFace** (la librería de bajo nivel). Lo pide la consigna y es lo que usa GPT-2. Partir de los 256 bytes garantiza que cualquier texto se pueda codificar sin un token `<unk>` (la ida y vuelta dio 5.000 de 5.000). Costo: unas 160 de esas 256 filas no se usan nunca en estos cuentos.

2. **V = 8.192**, el techo del rango que sugiere la consigna (4.096–8.192) y cerca de los ~10K del paper. Con este tamaño casi toda palabra es un token, las secuencias quedan cortas y más arriba ya no se gana nada. *Duda*: 4.096 podía ser igual de bueno para un modelo chico (ver la respuesta 3).
3. **Entrenar con los primeros 100.000 cuentos del split train**, el 4,7% de los 2.119.719. Tarda unos 10 segundos y alcanza: el tokenizador del barrido, entrenado con 5 veces menos cuentos (20.000), da casi lo mismo con **V = 8.192** (195,90 contra 195,75 tokens por cuento). *Duda*: son los primeros 100.000, no una muestra al azar; si el dataset tiene algún orden interno, podría haber un sesgo. No lo chequeamos.
4. **Medir sobre 5.000 cuentos del split validation**, que el tokenizador nunca vio. Medir sobre sus propios cuentos de entrenamiento sería hacernos trampa: un BPE comprime mejor lo que ya vio.
5. **Un único token especial, <|endoftext|>**, para marcar dónde termina un cuento y empieza el siguiente cuando se concatenan. No hace falta <unk>, porque todo byte tiene su token.
6. **add_prefix_space=False**, como GPT-2. Los cuentos arrancan pegados al <|endoftext|> que los separa, sin espacio, así que es coherente que su primera palabra no lleve **Ġ**. *Costo*: la misma palabra ocupa filas distintas según dónde aparece. **Once** (inicio de cuento) nace en el merge 169, porque 3.327 de los 5.000 cuentos arrancan con "Once upon a time" (anexo E); **Ġonce** recién en el 2.650 y **Ġonce** en el 5.344 (anexo B). Son tres filas para una misma palabra, y van a tener embeddings distintos.
7. **Comparar contra gpt2 y cl100k_base**, contando el costo como $2 \cdot V \cdot n_embd$ con $n_embd = 256$: dos matrices separadas, la de embeddings y la de salida. GPT-2 es el que pide la consigna; cl100k suma un tercer punto con un vocabulario gigante.
8. **Barrer V de 512 a 16.384 con tokenizadores entrenados sobre 20.000 cuentos**, para ver la curva entera y no un solo punto. Los 20.000 son para que los seis entrenamientos sean rápidos. *Consecuencia*: el punto 8.192 del barrido no coincide exacto con la tabla (195,90 contra 195,75 tokens por cuento, y 16,46% contra 15,55% de vocabulario sin usar; anexo C). Con 5 veces más datos, casi no cambia.
9. **Medir cuánto de un cuento entra en 128, 192 y 256 tokens**, como insumo para elegir **block_size**. Se eligió 256.
10. **SEED está definida pero no se usa, y no hace falta**. Entrenar un BPE no tiene azar: cuenta pares y une el más frecuente, y los cuentos son siempre los primeros N, no una muestra aleatoria. Lo comprobamos de tres formas: reentrenando por fuera salen exactamente los mismos números del barrido (anexo C); el **tokenizer.json** que generamos en otra computadora (Windows) es idéntico byte a byte, con el mismo SHA-1, al de esta corrida (Linux); y en otra corrida de este notebook lo único que cambió fue el tiempo (10,0 s contra 10,3 s) y el desempate de los tokens más largos.
11. **Corrió localmente** (Linux, con la RTX 5070 de una notebook) y no en Colab. Para esta etapa da igual: no usa GPU y tarda segundos.

Registro de decisiones de la Etapa 1

- **Con qué contábamos**: esta etapa es obligatoria, alimenta a todas las demás con **checkpoints/tokenizer.json** y es barata (segundos de CPU). No había nada que recortar por tiempo ni por memoria.
- **Qué hicimos**: todo lo que pide la consigna, más tres cosas que no pedía: **cl100k** como tercer tokenizador, el barrido completo de **V** y la medición de la ventana de contexto.
- **Qué no hicimos, y por qué**: el experimento que de verdad contesta qué **V** conviene, que es preentrenar con 4.096 y con 8.192 y comparar en bits por carácter. Cada valor de **V** implica repetir la Etapa 2 entera.
- **Qué haríamos distinto sabiendo lo que sabemos ahora**: tomar una muestra al azar en vez de los primeros cuentos, y mirar el texto crudo antes de entrenar. El texto mal codificado de la sección siguiente lo encontramos recién al analizar los resultados.

Dudas que nos quedan

- **El corpus trae texto mal codificado, y el tokenizador lo aprendió**. Cerca del 6% de los cuentos (302 de 5.000) tiene *mojibake*: en lugar del apóstrofo ' aparece â€™, por ejemplo en **Benâ€™s**. El BPE no distingue un error de un patrón, así que le dedicó tokens: **donâ€™t** se parte en **Ġdon** + **â€™** + **t**, mientras que **don't** es **Ġdon** + **'** + **t**

(anexo E). Para el modelo son dos maneras distintas de escribir la misma palabra, y puede aprender a generar las dos. ¿Tendríamos que haber limpiado el texto antes? Probablemente sí, porque es barato. No lo hicimos porque lo vimos recién ahora, cuando el tokenizador ya lo usan las etapas siguientes: limpiarlo obligaría a reentrenar todo. Es una grieta en la idea de TinyStories como un instrumento de laboratorio "limpio".

- **¿Está bien que casi todo sea palabra entera?** `Ġplay`, `Ġplayed`, `Ġplaying` y `Ġplays` son cuatro filas independientes. Con un vocabulario más chico compartirían el pedazo `Ġplay`, y lo que el modelo aprendiera de una forma le serviría para las otras. Con el nuestro, el modelo tiene que descubrir solo, por el contexto, que están relacionadas. La Etapa 3 puede contestarlo mirando si terminan cerca.
- **Los primeros 100.000 cuentos no son una muestra al azar.** No sabemos si el orden del dataset tiene alguna estructura, por ejemplo por cómo se generaron los cuentos. Si la tiene, el tokenizador podría estar un poco sesgado.
- **Una conexión que nadie nos pidió: SolidGoldMagikarp.** El repositorio de arranque de la cátedra, del que sale este proyecto, es de `solidgoldmagickarp`, y no creemos que el nombre sea casualidad. `SolidGoldMagikarp` es un token real del vocabulario de GPT-2, el 43.453 (anexo F). Es el nombre de un usuario de Reddit, y la explicación más aceptada es que aparecía mucho en los textos con los que se entrenó el tokenizador y casi nada en los datos del modelo. Su fila de embedding quedó prácticamente sin entrenar, y en 2023 se vio que los modelos que usaban ese vocabulario respondían cualquier cosa cuando se les pedía repetirlo. Es la versión extrema de las filas "sin usar" del barrido. En nuestro caso el riesgo es mucho menor, porque el tokenizador y el modelo se entrenan con el mismo corpus, pero no es cero: la cola larga del vocabulario existe igual.

Etapas 2 · Preentrenamiento y ablación ancho contra profundo

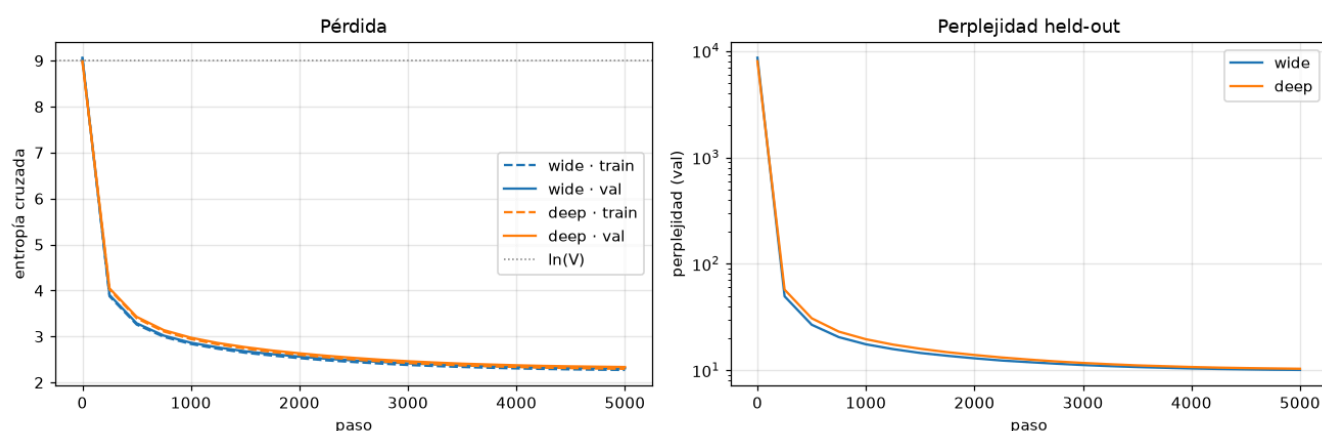


Figura 4. Pérdida en entrenamiento y validación, y perplejidad held-out, de los modelos ancho (256×2 capas) y profundo (192×5). La línea punteada es $\ln(V)$, la pérdida de adivinar al azar.

Este informe interpreta la corrida completa guardada en el notebook `02_pretraining` (el log de entrenamiento de cada modelo está en su celda) y su tabla de ablación. A lo largo del informe distinguimos estructura de oración de coherencia, menciones de uso correcto del contexto y resultados medidos de posibles explicaciones.

Respuestas conceptuales

¿Cuándo deja de ser ruido? Las primeras frases reconocibles aparecen con una pérdida de validación de aproximadamente **4,0 a 3,3**. En el paso 0, con pérdida cercana a 9, ambos modelos mezclan palabras y fragmentos sin conexión. A los 250–500 pasos aparecen estructuras de oración y frases como “She was happy.”, todavía rodeadas de errores. Entre pérdidas de **3,0 y 2,6** vemos más organización narrativa y, desde el paso 2.000, ambos sostienen el nombre Lily. Sin embargo, al terminar en **2,311 y 2,336** siguen confundiendo acciones, objetos y pronombres. La estructura local aparece antes que la consistencia; no encontramos un umbral que garantice cuentos coherentes ni gramática completamente estabilizada.

¿Se reproduce ancho–conocimiento y profundidad–contexto? Solo encontramos indicios parciales. Ambos modelos aciertan **2 de 10** prompts de recuerdo, aunque el ancho asigna más probabilidad media a las respuestas admitidas: **0,121 frente a 0,101**. El profundo obtiene más reuso de palabras clave, **0,58 frente a 0,52**, pero los textos muestran que repetir nombres no implica conservar identidades. La escala, el entrenamiento todavía en progreso y el diseño de los prompts podrían explicar la falta de una diferencia clara; no los identificamos como causas demostradas. Una sola semilla y pocos casos no permiten confirmar el paper ni declarar una superioridad general.

¿Qué se recortó? Se utilizaron 400.000 cuentos, dos configuraciones de unos 5–6 millones de parámetros, contexto de 256 tokens y 5.000 pasos por modelo. Se mantuvo la ablación, pero no se exploraron varias semillas, más arquitecturas ni una evaluación amplia. La corrida registrada usó una RTX 5070 Laptop con aproximadamente 8 GB de VRAM, precisión mixta y almacenamiento del corpus en `uint16` para reducir memoria. No consta que un fallo de memoria impusiera ese presupuesto. **El límite fue el tiempo, no la memoria.** El trabajo del grupo se concentró en los tres días previos a la entrega, y queríamos que esta etapa corriera en minutos para dejar lugar a las Etapas 3 a 5: el ancho tardó 6,3 minutos y el profundo 8,5. Con $5.000 \text{ pasos} \times 64 \text{ ventanas} \times 256 \text{ tokens} = 81,9 \text{ millones de tokens}$, 400.000 cuentos (88,5 millones) alcanzan para que cada modelo procese menos tokens de los que tiene el corpus (0,93 épocas): casi no ve dos veces lo mismo, y la brecha entre entrenamiento y validación queda en 0,04. Corrimos en la notebook con la RTX 5070 y no en Colab porque así el pipeline entero tardaba poco más de media hora, sin depender de sesiones que se cortan ni de montar Drive para no perder los checkpoints; los notebooks siguen preparados para Colab (primera celda). El registro de decisiones del proyecto está al final de este informe. Con más recursos priorizaríamos repeticiones y mejores pruebas de contexto antes de aumentar el tamaño.

¿Dónde quedamos frente a Chinchilla? Había **88,53 millones de tokens disponibles** en entrenamiento, pero cada modelo procesó **81,92 millones**: $5.000 \text{ pasos} \times 64 \text{ ventanas} \times 256 \text{ tokens}$. Las ventanas aleatorias pueden repetirse, por lo que el equivalente a 0,93 épocas no representa un recorrido ordenado del 93 % del corpus. Los tokens de validación no se usan para actualizar parámetros.

Contando todos los parámetros, procesamos aproximadamente **14,0 tokens por parámetro en el ancho y 15,1 en el profundo**. Eso representa el **70 % y 76 %** de la referencia de 20 tokens por parámetro. Podría haber quedado capacidad sin aprovechar, pero no podemos cuantificar cuánto rendimiento faltó. Si contamos solo los bloques, lo procesado equivale a **2,60 y 1,84 veces** esa referencia. El cambio se debe al enorme peso de embeddings y salida, no a que el mismo entrenamiento sea simultáneamente insuficiente y excesivo. La categoría “bloques” también excluye posiciones y normalización final. La regla es orientativa: Chinchilla estudia otras escalas y presupuestos, y superar ese cociente no demuestra sobreajuste ni desperdicio. [Hoffmann et al., 2022](#).

Diseño y comparación de los modelos

El experimento, en corto. *Hipótesis* (la del paper): con una cantidad de parámetros parecida, el modelo ancho debería recordar mejor datos comunes y el profundo sostener mejor, varias oraciones después, lo que apareció antes en el cuento. *Qué variamos*: solo el ancho del residual y la cantidad de capas (256×2 contra 192×5 ; las cabezas pasan de 8 a 6 para que cada una siga teniendo 32 dimensiones). *Qué dejamos fijo*: tokenizador, datos, orden de los batches, semilla, contexto, optimizador y presupuesto de pasos, detallados abajo. *Cómo lo medimos*: pérdida y perplejidad sobre *validation*, diez prompts de recuerdo (acierto greedy y probabilidad de la respuesta), cinco prompts de consistencia con cinco muestras cada uno, y la lectura de las muestras. Contamos los parámetros antes de entrenar: el profundo tiene un 7,2 % menos en total, pero un 41 % más en los bloques.

El modelo aprende a predecir el próximo token a partir de los anteriores. El BPE de la Etapa 1 define las **8.192 unidades** con las que representamos el texto; el preentrenamiento aprende sus combinaciones y dependencias. La pérdida recompensa acertar el siguiente token, sin evaluar directamente si la historia completa tiene sentido.

Cada cuento termina con `<|endoftext|>` y se concatena con el siguiente. El delimitador señala límites y puede detener la generación, pero no impide atender al cuento anterior dentro de una ventana. Elegimos 256 tokens porque el **90,2 %** de los cuentos medidos en la Etapa 1 cabía por longitud, frente al 60,4 % con 192. Ese porcentaje se midió sobre cuentos de *validation*, y los de *train* son algo más largos (221 tokens por cuento contra 201, contando el `<|endoftext|>`, según la tokenización del notebook): entre los datos de entrenamiento, la fracción que entra es menor. Además, las ventanas comienzan al azar: pueden cortar cuentos o mezclar el final de uno con el principio de otro.

Los modelos comparten datos, orden de batches, semilla 1337, contexto, dropout de 0,1 y presupuesto. Usan AdamW, calentamiento de 100 pasos, tasa de aprendizaje de $3e-4$ con descenso coseno hasta $3e-5$, weight decay de 0,1 sobre matrices y recorte de gradiente de 1,0. La validación utiliza un split separado y 50 batches fijos por evaluación, sin dropout, para comparar la evolución sobre las mismas ventanas.

Configuración o resultado	Ancho	Profundo
Dimensión / capas	256 / 2	192 / 5
Cabezas × dimensión por cabeza	8 × 32	6 × 32
Parámetros totales	5.846.528	5.424.896
Embeddings de tokens y posiciones	2.162.688	1.622.016
Bloques Transformer	1.577.984	2.221.440
Salida y normalización final	2.105.856	1.581.440
Pérdida de entrenamiento	2,273	2,297
Pérdida de validación	2,311	2,336
Perplejidad de validación	10,08	10,34
Recuerdo: aciertos greedy	2/10	2/10

Configuración o resultado	Ancho	Profundo
Recuerdo: probabilidad media admitida	0,121	0,101
Reuso de palabras clave	0,52	0,58
Tiempo registrado	6,34 min	8,52 min

Embeddings y salida no comparten pesos: sus matrices principales crecen como vocabulario $\times$ ancho, mientras que los bloques crecen aproximadamente como capas $\times$ ancho². Por eso reducir el ancho a la mitad y duplicar las capas no conserva parámetros. Nuestro profundo tiene **7,2 % menos parámetros totales**, pero **41 % más en los bloques**. Embeddings y salida concentran aproximadamente el 73 % del ancho y el 59 % del profundo: también cambia dónde se distribuye la capacidad.

Igualar pasos y tokens tampoco iguala costo: el profundo tardó aproximadamente **34 % más**. El tiempo registrado incluye evaluaciones y tareas intercaladas, no solo actualizaciones. El ancho obtiene una ventaja de **0,025 en pérdida de validación** y aproximadamente **2,5 % en perplexidad**, válida para esta corrida, sin demostrar superioridad general.

Curvas y muestras: qué mejora y qué falta

La entropía cruzada penaliza asignar poca probabilidad al token correcto; la **perplexidad** = $\exp(\text{pérdida})$ expresa esa misma medición en otra escala. Los valores iniciales, 9,063 y 8,988, están cerca de $\ln(8.192) \approx 9,01$, la referencia de una predicción uniforme. A los 500 pasos, las perplexidades ya bajan a 26,8 y 30,8. Después las mejoras se vuelven menores.

La distancia final entre entrenamiento y validación es de aproximadamente **0,038–0,039**. No vemos una subida de validación mientras entrenamiento sigue bajando: no hay una señal clara de sobreajuste en estas curvas. Tampoco demostramos convergencia. Comparando el mismo tramo, del paso 4.250 al 5.000, validación baja **0,020 en el ancho y 0,025 en el profundo**. Ambos seguían mejorando; la desaceleración también coincide con la reducción programada de la tasa de aprendizaje.

Las muestras permiten interpretar esa mejora sin confundirla con calidad narrativa:

- **Paso 500:** el ancho combina “She was happy.” con “Jane was a would go home.”. El profundo escribe “a little girl named she loved to play”: reconoce una fórmula, pero no completa correctamente el nombre.
- **Paso 1.000:** aparecen oraciones como “She saw a big trunk and melted.” en el ancho y “she saw a voice” en el profundo. La estructura mejora antes que el sentido.
- **Paso 2.000:** ambos conservan a Lily, pero el ancho cierra con “promised to never peanut on things again” y el profundo con “she went to play together and died.”. Aprender una fórmula de cierre no equivale a resolver la historia.
- **Paso 5.000:** el ancho mantiene muñecas y ropa, pero introduce referencias ambiguas y “she put it on harder”. El profundo hace que un búho llame “bird” a Lily y luego confunde pronombres. Los finales truncados deben interpretarse considerando el límite de **120 tokens nuevos**: un corte no demuestra por sí solo incapacidad para terminar.

La semilla fija controla el muestreo, pero una trayectoria no representa toda la calidad del modelo. Palabras como “harder” o “peanut” reaparecen entre muestras; compartir el generador aleatorio puede contribuir, aunque no probamos esa causa. También observamos “” en una continuación. La Etapa 1 midió que el 6 % de los cuentos held-out tiene texto mal codificado de esa misma familia (“: el ‘ guardado como “”, el “ como “”), así que lo más probable es que el modelo lo haya aprendido del corpus; no rastreamos qué cuentos de entrenamiento lo originan.

Qué miden realmente los prompts

En recuerdo, **greedy** elige el token más probable y se verifica la primera palabra generada. La probabilidad admitida suma, en cambio, la masa de los tokens aceptados como respuesta inmediata. El ancho acierta pez $\rightarrow$ “sea” y fuego $\rightarrow$ “hot”; el profundo, pez $\rightarrow$ “pond” y aves $\rightarrow$ “fly”. La probabilidad permite distinguir asociaciones que el empate 2/10 oculta: para fuego da **0,186 frente a 0,058**, mientras que para aves favorece al profundo, **0,245 frente a 0,079**.

El conjunto de respuestas condiciona el resultado: para el pez se admiten seis palabras, pero para el cielo solo “blue”, por lo que “dark” cuenta como error aunque sea posible. La probabilidad solo contempla respuestas de un token con espacio inicial. Además, los prompts son fragmentos de texto presentados como inicio de cuento, no preguntas explícitas. La métrica mide esas continuaciones y respuestas admitidas, no conocimiento factual general.

En consistencia hay **cinco prompts × cinco muestras = 25 generaciones por modelo**, no 25 prompts diferentes. Con dos palabras clave, el reuso equivale a **26 frente a 29 menciones sobre 50 posibles**. Se cuenta presencia al menos una vez, sin exigir reaparición tardía o uso correcto. Las salidas impresas muestran la primera generación de cada prompt; bastan para detectar límites de la métrica:

- Ambos alcanzan 100 % con Mia y la pelota, aunque el ancho termina haciendo que Mia huya de Mia.
- El profundo logra 100 % con Tom y Max mientras escribe “Hi, I’m Tom,” Max said: repetir ambos nombres encubre una confusión de identidad.
- Ambos mencionan a Ben y el castillo, pero las acciones se desvían hacia objetos desconectados. En el caso de Pip y la pluma, ambos abandonan las palabras clave.

También podría haber continuidad correcta mediante pronombres sin repetir un nombre. Por eso el reuso puede premiar incoherencias y penalizar continuidades válidas. La diferencia de tres menciones es una tendencia descriptiva; no estimamos significancia ni variabilidad entre corridas.

Como evidencia complementaria, el juez de la Etapa 5 da consistencia **2,17 al ancho y 1,96 al profundo**, en dirección opuesta al reuso. Evalúa **24 textos por modelo, con otros prompts y generaciones**, y presenta un efecto piso en gramática: tampoco es una medida definitiva. Sus perplejidades, **10,32 y 10,58**, corresponden a otra evaluación con otros batches; no reemplazan las **10,08 y 10,34** de esta etapa ni indican entrenamiento adicional de los modelos base.

Conclusión y experimento pendiente

El resultado central es que **mejorar la predicción local puede convivir con historias incoherentes**. En validación el modelo recibe contexto real; al generar, continúa sus propias decisiones y errores. Puede conservar frases típicas y nombres mientras pierde quién posee un objeto o por qué ocurre una acción. Los ejemplos de Mia y Max muestran por qué una métrica de repetición da una impresión engañosa de memoria.

Podemos sostener que ambos modelos aprendieron estructura lingüística y que el ancho obtuvo una pequeña ventaja predictiva con menor tiempo de ejecución. Sigue abierto si la profundidad favorece el contexto bajo más entrenamiento y una evaluación adecuada.

Como experimento futuro, repetiríamos el par con al menos tres semillas y relatos que introduzcan una relación explícita —quién guarda una llave y dónde—. Evaluaríamos su recuperación tras 32, 64 y 128 tokens, controlando el texto intermedio y manteniéndolo dentro del contexto. Combinaríamos aciertos, probabilidades y lectura de continuaciones. Así podríamos medir si la ventaja depende de la distancia al dato y se sostiene entre corridas. Esta prueba está propuesta, no realizada.

Etapa 3 · La representación aprendida

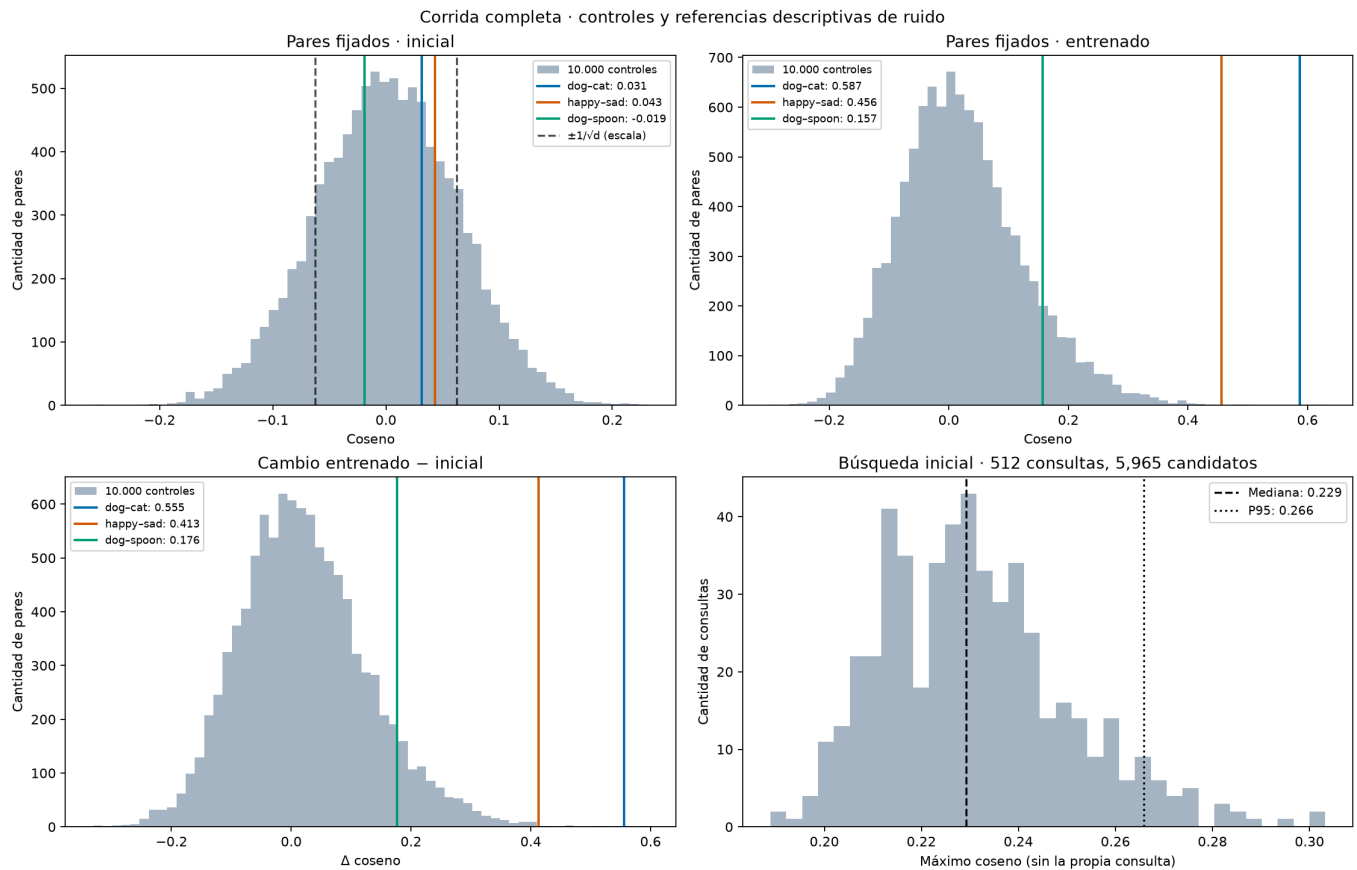


Figura 5. Pares elegidos contra 10.000 pares de control al azar: coseno inicial, coseno entrenado y cambio; abajo a la derecha, el máximo coseno de una búsqueda en el modelo sin entrenar.

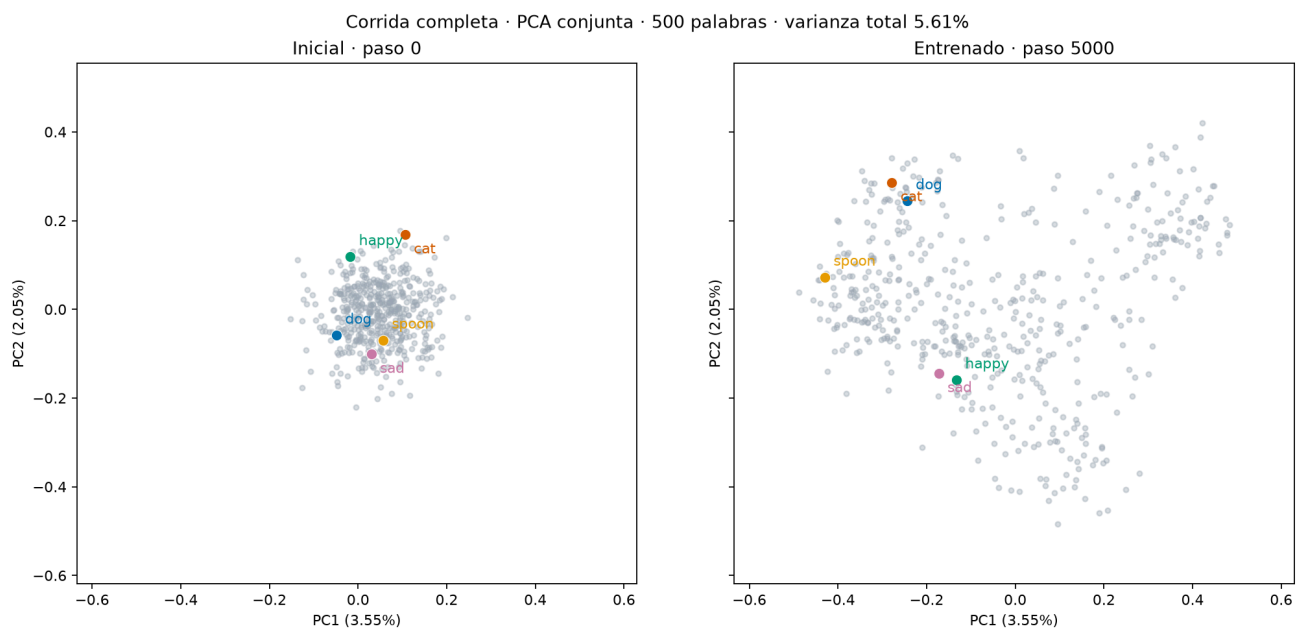


Figura 6. PCA conjunta de 500 palabras enteras, paso 0 contra paso 5.000, con los mismos ejes.

Respuestas a las cuatro preguntas

Analizamos el **modelo ancho de la Etapa 2, del paso 0 al paso 5.000**, con embeddings de 256 dimensiones. En las salidas guardadas, el paso 5.000 coincide con la mejor pérdida de validación del historial disponible: **2,310681**. Las cifras que discutimos abajo provienen de esta corrida completa, no del modo `SMOKE_TEST`.

1. ¿Los pares elegidos quedaron cerca o lejos? ¿El cambio se distingue del ruido?

Los dos pares que esperábamos cercanos se acercaron mucho más que la mayoría de los controles. La expectativa para **dog-spoon** se sostiene en términos relativos. Resumimos los cosenos y sus cambios, redondeados a tres decimales; el cambio se calculó antes de redondear.

Par	Expectativa	Coseno inicial	Coseno entrenado	Cambio
dog-cat	Cerca	0,031	0,587	+0,555
happy-sad	Cerca	0,043	0,456	+0,413
dog-spoon	Más lejos	-0,019	0,157	+0,176

Los histogramas ayudan a poner esos números en contexto. Antes de entrenar, los 10.000 pares aleatorios se concentran alrededor de cero y nuestros tres pares están dentro de esa zona. Después, la distribución de controles se ensancha y su media pasa de 0,000841 a 0,022445. Por eso no alcanza con observar que un coseno aumentó: también cambió la geometría del conjunto. En el histograma de cambios, la mediana de los controles es 0,013 y el percentil 95 es 0,215; los aumentos de **dog-cat** y **happy-sad** quedan muy por encima de esas referencias. Sus cambios están en los percentiles empíricos 99,99 y 99,89, respectivamente. Esto respalda una reorganización especialmente marcada para estos pares, aunque no demuestra que el resto del espacio haya quedado igual.

dog-spoon termina menos cerca que los otros dos pares, pero su aumento de 0,176 no es nulo ni despreciable. El coseno final queda en el percentil 89,85 de los controles y el cambio en el 91,62: ambos están por debajo de sus respectivos percentiles 95. Nuestra predicción funciona si «lejos» significa **menos similar que los pares relacionados**, no si significa ausencia de relación. Podría influir una reorganización más general o algún contexto compartido en los cuentos; estas mediciones no permiten decidir cuál explicación pesa más.

Los vecinos aportan una lectura concreta. Al inicio, cerca de **dog** aparecen **germs**, **danger** y **stay**; después aparecen **cat** (0,5867), **puppy** (0,5520), **pup** y **wolf**. Para **cat**, los primeros vecinos incluyen **dog**, **mouse**, **puppy** y **kitten**. Vemos un entorno más reconocible de animales, aunque la aparición de **monster** entre los vecinos de **dog** recuerda que la organización no es una clasificación de diccionario. Algo parecido ocurre con **spoon**: sus vecinos entrenados incluyen **fork** (0,5153), **pan**, **cup** y **bowl**, junto con objetos como **wand** y **pen**. Una explicación posible es que compartan acciones o posiciones dentro de las narraciones, además de categorías de objetos.

Para **happy** aparecen **glad** (0,5232), **relieved** y **excited**; para **sad**, **upset** (0,6575), **frustrated** y **miserable**. A la vez, **sad** queda séptimo entre los vecinos de **happy**. No lo leemos como una confusión entre alegría y tristeza: los antónimos pueden ocupar contextos parecidos, por ejemplo al describir cómo se siente un personaje. **La cercanía distribucional no equivale a sinonimia**. Es una explicación compatible con los resultados; para comprobar los contextos concretos tendríamos que inspeccionar ocurrencias del corpus.

El cuarto histograma explica por qué incluso el modelo inicial tiene vecinos con cosenos que parecen altos. Para un par fijado antes de mirar, la escala orientativa es $1/\sqrt{256} = 0,0625$. Pero al buscar el máximo entre 5.965 candidatos, seleccionamos un extremo: en las 512 consultas iniciales, la mediana de esos máximos es 0,229 y su percentil 95 es 0,266. Un vecino inicial con coseno cercano a 0,2 puede surgir por esa búsqueda entre miles de opciones; por sí solo no acredita aprendizaje. Los cosenos finales de nuestros dos pares relacionados superan ampliamente esas referencias iniciales y también el percentil 99 de los cosenos entrenados de control (0,304).

Usamos todos estos percentiles como **referencias descriptivas**, no como p-valores ni pruebas concluyentes de significado. Los controles pueden compartir palabras y algunos también pueden estar relacionados. La evidencia más convincente para nosotros es la coincidencia entre cambios grandes frente a los controles y vecinos que podemos interpretar, con esas limitaciones.

2. ¿Qué cambió en la proyección 2D?

En el panel inicial vemos una nube compacta, con las consultas repartidas sin una separación temática clara. En el panel entrenado la nube ocupa una región más amplia y los pares relacionados se acercan visiblemente: **dog** y **cat** casi se superponen en la parte superior izquierda, mientras que **happy** y **sad** quedan próximos en la zona inferior. **spoon** queda

separado de esos pares. Esa lectura coincide, para los pares relacionados, con el aumento de coseno medido en las 256 dimensiones.

La comparación es útil porque proyectamos las mismas 500 palabras con **una PCA ajustada conjuntamente** sobre ambos estados. Así, los ejes representan las mismas direcciones; dos PCA independientes podrían rotar las vistas de manera distinta. Compartir límites y escala también evita que un cambio de zoom explique la mayor dispersión aparente. Como usamos vectores normalizados, esa dispersión en el plano tampoco se debe simplemente a que hayan aumentado sus normas.

El límite es importante: la PC1 explica el 3,55 % de la varianza, la PC2 el 2,05 % y el total guardado, calculado antes del redondeo, es **5,61 %**. El 94,39 % restante es variación omitida por estos dos ejes, no una cantidad medible de «significado perdido». La PCA selecciona direcciones de variación, no mide cuánto significado contiene cada una. Además, ese porcentaje corresponde a la muestra conjunta utilizada, no a una medición de todo el vocabulario.

Por eso no concluimos que el gráfico revele todos los grupos semánticos ni que sus distancias reproduzcan las originales. La mayor dispersión en estos ejes tampoco demuestra por sí sola que todos los tokens se hayan alejado entre sí. Usamos la figura para explorar y los cosenos originales para sostener la comparación cuantitativa.

3. ¿Qué cambia al filtrar palabras?

El filtro reduce el vocabulario de 8.192 tokens a 5.966 candidatos que empiezan con **Ġ** y continúan con letras. La consulta sigue siendo la misma fila **Ġpalabra**; lo que cambia es el conjunto donde buscamos sus diez vecinos. **Sin filtrar**, la proporción de vecinos que pasa el criterio es:

Consulta	Inicial	Entrenado
dog	70 %	100 %
cat	60 %	100 %
happy	90 %	100 %
sad	80 %	100 %
spoon	90 %	100 %

Al inicio, filtrar hace las listas más legibles: elimina, por ejemplo, **ving** y **ee** entre los vecinos de **dog**, y **onies** y **sm** entre los de **cat**. Después del entrenamiento, los diez vecinos de cada una de estas cinco consultas ya pasan el filtro, de modo que ambas búsquedas devuelven las mismas listas. Esto sugiere una organización local más interpretable para **estas consultas observadas**; no demuestra que ocurra en todo el vocabulario ni que los fragmentos carezcan de información. El 100 % de las listas filtradas, en cambio, está garantizado por la selección y no es un resultado de aprendizaje.

Llamar «palabras enteras» a los candidatos es una simplificación operativa. **ĠBu**, presente entre los vecinos iniciales de **dog**, pasa el filtro aunque puede ser un comienzo de palabra; el criterio no verifica una entrada de diccionario ni su uso en contexto. Además, ampliar los candidatos puede cambiar los mejores vecinos por el propio tamaño de la búsqueda, sin que haya cambiado ningún embedding.

El ejemplo **Ġday** frente a **day** muestra otro límite: tienen IDs distintos, 356 y 1112, y solo el primero pasa el filtro. El marcador **Ġ** representa un espacio previo en esta tokenización; la fila sin marcador puede corresponder a otras posiciones o fragmentos. Su coseno pasa de $-0,136445$ a $0,163688$: no son vectores intercambiables pese a compartir las letras. Mantener los IDs evita consultar otra fila al quitar el marcador. No medimos sus frecuencias de uso, así que no podemos afirmar que una variante apareció menos ni que quedó sin entrenar.

4. Si el espacio siguiera pareciéndose al inicial, ¿qué cambiaríamos y cuánto costaría?

En esta corrida **sí observamos cambios** en los cosenos, los vecinos y la proyección. Aun así, estudiamos embeddings estáticos de una sola corrida: cada token tiene aquí un vector fijo, mientras que las capas posteriores construyen representaciones que dependen del contexto. Un resultado débil en esta tabla no demostraría por sí solo ausencia de aprendizaje en todo el modelo.

Si la evidencia siguiera siendo débil, primero revisariamos que estemos comparando los checkpoints correctos y las mismas filas del tokenizador. Después considerariamos las siguientes alternativas, como **experimentos futuros que no realizamos en esta etapa**:

- **Más pasos con la misma configuración y datos:** permitirían comprobar si faltó entrenamiento, a costa de más cómputo y posible sobreajuste. Mirariamos la pérdida de validación junto con los cambios de embeddings.
- **Más datos pertinentes y diversos, manteniendo el tokenizador:** permitirían probar una cobertura mayor de usos, con costos de preparación, tokenización, almacenamiento y entrenamiento. Más variedad no garantiza mejorar estas relaciones; también puede cambiar qué contextos predominan.
- **Otra dimensión de embedding:** permitiría estudiar si la capacidad limita las representaciones, pero cambia parámetros, memoria y costo por paso. Una dimensión mayor no garantiza una mejora y modifica la escala de ruido $1/\sqrt{d}$, por lo que no reutilizaríamos los umbrales de esta corrida.
- **Otras semillas con la misma configuración:** servirían para evaluar robustez, no para prometer mejores vecinos. Su costo es repetir entrenamientos y análisis.

Cambiaríamos una variable por vez y conservaríamos una referencia del paso 0 para cada nueva corrida. Mantendríamos consultas y criterios de evaluación, recalculariamos controles y registraríamos tiempo, memoria y validación. Así podríamos distinguir una mejora reproducible de una diferencia propia de la inicialización o de la medición.

Decisiones, diseño y reflexión del grupo

Qué nos permite el recorte realizado

Decisión	Ventaja para esta pregunta	Límite de la decisión
Reutilizar el modelo ancho de la Etapa 2	Conecta el análisis con un modelo ya entrenado y permite inspeccionar su tabla sin entrenar otro.	Las conclusiones corresponden a esa configuración y corrida; no permiten atribuir el resultado al ancho ni compararlo con otras arquitecturas.
Comparar el paso 0 con el 5.000	Contrasta la inicialización con un estado entrenado que, aquí, coincide con la mejor validación registrada.	Dos puntos no muestran cuándo surgieron las relaciones ni garantizan que 5.000 sea el óptimo fuera del historial disponible.
Leer la tabla en CPU	Evita reconstruir y ejecutar el Transformer para calcular cosenos y PCA; facilita volver a inspeccionar los artefactos.	No evalúa generación ni representaciones contextuales. Es una ventaja del análisis elegido, no evidencia de una restricción histórica de hardware.
Normalizar los vectores	Permite comparar direcciones con similitud coseno sin que domine la magnitud de cada fila.	Dejamos fuera los posibles cambios de norma; describimos una parte de la geometría aprendida.
Usar los mismos controles y consultas	Hace comparables los cambios y evita que una nueva selección explique las diferencias entre estados.	Los pares aleatorios no son una muestra de relaciones garantizadas como «sin significado» y las cinco consultas cubren pocos casos.

Estas son las ventajas y los costos conceptuales del recorte que podemos defender con los artefactos. No necesitamos atribuirlo a falta de tiempo o de GPU que este notebook no documenta.

Qué pusimos a prueba

Nuestra hipótesis era que `dog-cat` compartiría usos como animales y personajes, y `happy-sad` como estados emocionales, mientras que `dog-spoon` resultaría menos similar. Los pares estaban fijados antes de observar las salidas. Mantuvimos el tokenizador y sus IDs, la arquitectura, la dimensión, las consultas, los controles y los criterios de análisis. Para la figura mantuvimos también la muestra de palabras y los ejes conjuntos. Lo que comparamos son los pesos de la tabla antes y después del preentrenamiento; durante ese proceso también aprendieron los demás parámetros del modelo, que aquí no analizamos.

Medimos cosenos, cambios de coseno y diez vecinos, con pares aleatorios y máximos iniciales como referencias distintas. La PCA agrega una vista exploratoria. Esta comparación sostiene que las relaciones observadas cambiaron durante el entrenamiento, pero no identifica qué cuentos, frecuencias o mecanismos causaron cada cercanía.

La comparación supone que los dos checkpoints y el tokenizador pertenecen a la misma corrida. Lo respaldan dos cosas: las configuraciones de modelo y de entrenamiento coinciden, y los dos checkpoints llevan la misma huella SHA-1 del tokenizador, que es la de `checkpoints/tokenizer.json`. Lo que no guardan es la semilla: la 1337 que imprime este notebook fija el muestreo del análisis y no certifica por sí sola la inicialización del entrenamiento (la Etapa 2 la fija en su código, con `torch.manual_seed(SEED)` antes de construir cada modelo).

De las unidades del tokenizador a las capacidades del modelo

La conexión entre las tres primeras etapas se ve en las filas que analizamos. **El tokenizador define las unidades y el preentrenamiento modifica sus relaciones.** Tener `Ġdog` y `Ġcat` como tokens permite comparar directamente sus vectores; otras palabras pueden quedar repartidas en fragmentos. El caso de `Ġday` y `day` muestra por qué no podemos hablar de «la representación de una palabra» sin precisar qué token consultamos. Al aprender a predecir continuaciones, el modelo puede acercar unidades que aparecen en usos parecidos; nuestros vecinos son evidencia compatible con ese proceso, no una inspección directa de todos esos usos.

Nuestra reflexión central es que **reconocer relaciones entre palabras no garantiza sostener un cuento coherente ni seguir instrucciones.** Que `dog` esté cerca de `cat` ayuda a interpretar una relación local, pero no dice si el modelo mantendrá al mismo animal como protagonista, recordará qué hizo ni cerrará la historia sin contradicciones. Del mismo modo, que `happy` tenga vecinos emocionales no demuestra que el modelo vaya a incluir esa palabra cuando se la pedimos: debe usar la instrucción y el contexto al generar una secuencia.

Etapla 4 · SFT: comportamiento, no conocimiento

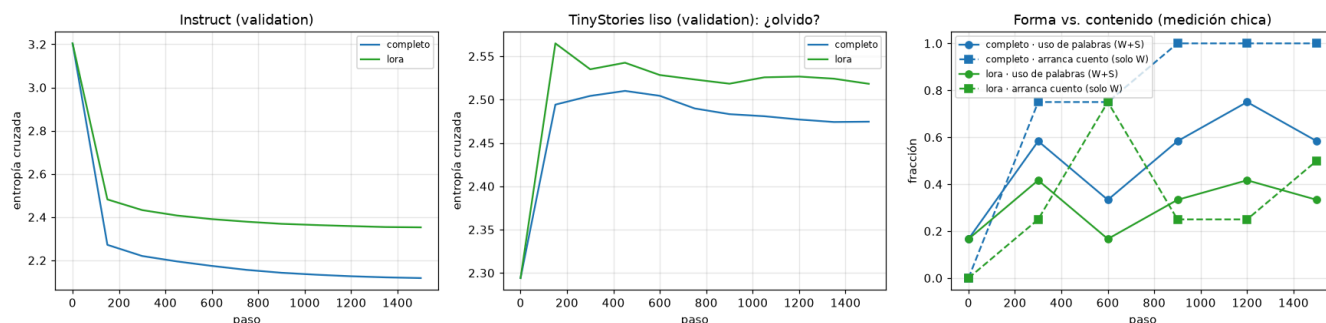


Figura 7. SFT completo y LoRA: pérdida en Instruct, pérdida en TinyStories liso (olvido) y forma contra contenido en la medición chica durante el entrenamiento.

Corrida local (RTX 5070 Laptop, 8 GB), todas las etapas en orden, semilla 1337. Base: el modelo **ancho** de la Etapa 2 ($n_embd=256$, $n_layer=2$, 5,85M parámetros, 5.000 pasos, val loss 2,31 / ppl 10,1). Todas las cifras de abajo salen de checkpoints/stage4/. Hay **una sola semilla por configuración**: los intervalos de confianza reflejan la variación entre prompts, no entre corridas.

Qué se entrenó:

- **SFT completo:** 1.500 pasos, batch 64, lr $1e-4 \rightarrow 1e-5$, sobre 100.000 registros de TinyStories-Instruct, cada bloque alineado al principio (instrucción + Story: + cuento). Tardó 1,7 min.
- **SFT LoRA:** mismo presupuesto, lr $1e-3$, rango 8 y $\alpha=16$ sobre `Head.key/query/value` y `MultiHeadAttention.proj`. El resto del modelo quedó congelado.
- **Medición:** 8 prompts de 3 palabras $\times$ 5 muestras, en las dos plantillas (Words + Story, que mide sobre todo contenido, y solo Words, que mide sobre todo forma).

1. El par antes/después: cambiaron las dos cosas, pero no por igual

Words: dragon, happy, forest, misma semilla para los tres modelos. Fragmentos textuales.

Plantilla solo Words (el prompt termina en forest):

Base: , and it would be so loud that day he was always bigger than the other animals were talking. One day Sam's parents said to him, "Let's go and see where the other person can find my secret from here!" ...

SFT completo: , good / Summary: Tim learns to share a bigger map by making his friend happy ending. / Story: Once upon a time, in a big forest, there was a green forest. On one last, he met a small rabbit. The rabbit felt happy and grateful. ...

SFT LoRA: , good / Summary: Tim observes a game where he needed food for the dragon ... / Summary: ... / Dort: Timmy remembers ... (no llega a escribir Story:)

Plantilla Words + Story:

Base: "Mr Jones! Are you so big? What's wrong?" Mama: "I'll bring to the dragon inside the candle". Slowly, the dragon kept walking away ... — usa 1 de 3 palabras.

SFT completo: Once upon a time, there was a little bear named Benny. Benny loved to hop around the forest near his home. ... Benny was happy ... — usa 2 de 3.

SFT LoRA: Lily loved to hide with her friend, Timmy. ... — usa 1 de 3 (happy).

La medición completa (`forma_contenido.csv`, `contenido_bootstrap.csv`):

	arranca cuento (solo Words)	termina (W+S)	uso de palabras (W+S)	piso de azar	uso – piso [IC95]
base	0 %	92,5 %	19 %	8 %	0,11 [0,00, 0,19]
SFT completo	92,5 %	92,5 %	52,5 %	21 %	0,32 [0,14, 0,46]
SFT LoRA	37,5 %	85 %	36 %	11 %	0,25 [0,17, 0,33]

- **La forma cambió de golpe y por completo.** Con solo la línea `Words:`, el base nunca escribió un cuento: sigue la frase como si `forest` fuera parte de una oración (, and it would be so loud...). El SFT completo, en el 92,5 % de los casos, trata esa línea como lo que es, un encabezado: primero completa la lista (, good), después agrega otro encabezado (`Summary:`), escribe `Story:` y arranca el cuento. Eso no es conocimiento nuevo sobre dragones o bosques: es haber aprendido la estructura del formato.
- **En `Words + Story` la forma ya estaba antes.** El base arranca un cuento el 92,5 % de las veces cuando el prompt termina en `Story:`, porque el preentrenamiento ya le enseñó a escribir cuentos. Lo que el SFT agrega en esa plantilla es contenido, no forma.
- **El contenido se movió, pero a medias.** El uso de las palabras pedidas pasó de 19 % a 52,5 %. La ganancia sobre el piso de azar (0,32) casi triplica la del base (0,11), y el intervalo del base toca el cero: el base solo "usa" palabras frecuentes en TinyStories (`happy`, `forest`) por casualidad. Aun así, el SFT completo usa en promedio la mitad de las palabras, no las tres.
- **La forma se movió primero.** En el panel derecho de las curvas, "arranca cuento" ya estaba en 75 % en el paso 300 (de 1.500) y en 100 % desde el 900, subiendo de manera monótona; el uso de palabras osciló entre 0,33 y 0,75 sin tendencia limpia.
- **En solo `Words` se movió una dimensión y la otra no.** Con esa plantilla, uso – piso da 0,22 en el base, 0,24 en el completo y 0,24 en LoRA: el contenido quedó igual mientras la forma pasaba de 0 % a 92,5 %. Es exactamente el caso que anticipa la consigna. (La comparación es aproximada: cuando un modelo no escribe `Story:`, como el base, "el cuento" es toda la continuación; cuando lo escribe, solo lo que viene después.)
- **Un número de la tabla que no hay que creerle.** `contenido_bootstrap.csv` dice que el base "le gana al azar", pero la cota inferior de su intervalo es $3,5 \cdot 10^{-18}$: un cero con error de redondeo de punto flotante. Leemos ese intervalo como [0,00, 0,19], que toca el cero.
- **Efecto secundario.** El SFT a veces mete líneas de encabezado dentro del cuento: 0,6 por generación en el completo (0,25 en LoRA), 0 en el base. Aprendió tan bien el formato que a veces lo sobreaplica.

En una línea: el SFT movió primero y del todo la forma — reconoce la instrucción y la convierte en un cuento. El contenido se movió menos: usa más de las palabras pedidas que el azar, pero no todas.

2. Qué falta para conseguir la parte que no se movió

1. **Enmascarar la pérdida de la instrucción**, para que todo el gradiente vaya al cuento condicionado. Hoy el modelo también aprende a generar encabezados, y eso explica el , good y los `Summary:` espontáneos.
2. **Filtrar a registros con `Words:`**. Solo una parte de Instruct tiene esa línea, así que la señal de "usá estas palabras" es más rala que la de formato, que está en todos los registros.
3. **Más pasos:** el uso de palabras no había convergido.
4. **Un modelo más profundo.** Usar una palabra 50-100 tokens después de leerla es un problema de contexto, y con 2 capas hay poco margen.

3. LoRA: qué fracción se entrenó y cuánto de la mejora compró

`costo_lora.csv`:

	completo	LoRA
parámetros entrenables	5.846.528 (100 %)	118.784 (1,99 %)
gradientes + AdamW (analítico)	70,2 MB	1,4 MB
memoria pico GPU al entrenar	3.065 MB	2.922 MB (-4,6 %)
segundos por paso	0,067	0,062 (-7 %)

comparacion_lora.csv, con la mejora medida como $(\text{LoRA} - \text{base}) / (\text{completo} - \text{base})$:

métrica	base	completo	LoRA	LoRA / completo
pérdida en Instruct (val)	3,20	2,12	2,35	78 %
contenido: uso – piso	0,11	0,32	0,25	68 %
forma: arranca cuento (solo Words)	0 %	92,5 %	37,5 %	41 %

- **Con el 2 % de los parámetros, LoRA compró el 78 % de la mejora en pérdida** y el 68 % de la de contenido. En comportamiento visible compró mucho menos: el 41 % de la forma.
- **La forma es lo que más le cuesta a LoRA, y tiene sentido.** Para escribir Story: en el momento justo hay que cambiar qué token es probable en un punto dado, y eso vive en el MLP y en `lm_head`, que LoRA dejó congelados; LoRA solo puede cambiar cómo se mira el contexto. En el par de arriba se ve: LoRA aprendió a producir encabezados (Summary:), pero se queda dando vueltas en ellos sin llegar a Story: . Variantes para probar: aplicar LoRA también al MLP, o descongelar `lm_head`.
- **La memoria casi no bajó (-4,6 %).** Los gradientes y el estado de AdamW del modelo completo ocupan 70 MB, y el pico pasa los 3 GB. A esta escala dominan las activaciones, sobre todo los logits: $64 \times 256 \times 8.192$ posiciones por batch, en fp16 más su copia en fp32 para la pérdida. LoRA reduce exactamente la parte que acá no pesa; su ventaja de memoria aparece en modelos donde los pesos y el optimizador dominan el pico, no en uno de 6M de parámetros.

4. Olvido

La pérdida en TinyStories liso subió de 2,29 a 2,47 con el SFT completo y a 2,52 con LoRA. En la Etapa 5, con otros batches, dio 2,33 → 2,57 y 2,54. El olvido existe y es moderado, pero **el orden entre completo y LoRA se invierte según la medición**, así que la diferencia entre los dos métodos está dentro del ruido: con este presupuesto no se puede afirmar que LoRA olvide menos, aunque no toque los pesos base.

Limitaciones

- Una sola semilla de entrenamiento por configuración: no hay variabilidad entre corridas.
- 40 generaciones por celda de la tabla. Alcanza para las diferencias grandes (forma), no para las de décimas.
- El 40 % de los registros de Instruct no entra en `block_size + 1 = 257` tokens y pierde el final del cuento (mediana de 244 tokens por registro, p90 de 372). El SFT ve muchos cuentos sin su cierre; igual el completo termina el 92,5 % de sus cuentos, lo mismo que el base, que lo había aprendido en el preentrenamiento.

Etapa 5 · Medir la emergencia con un juez local

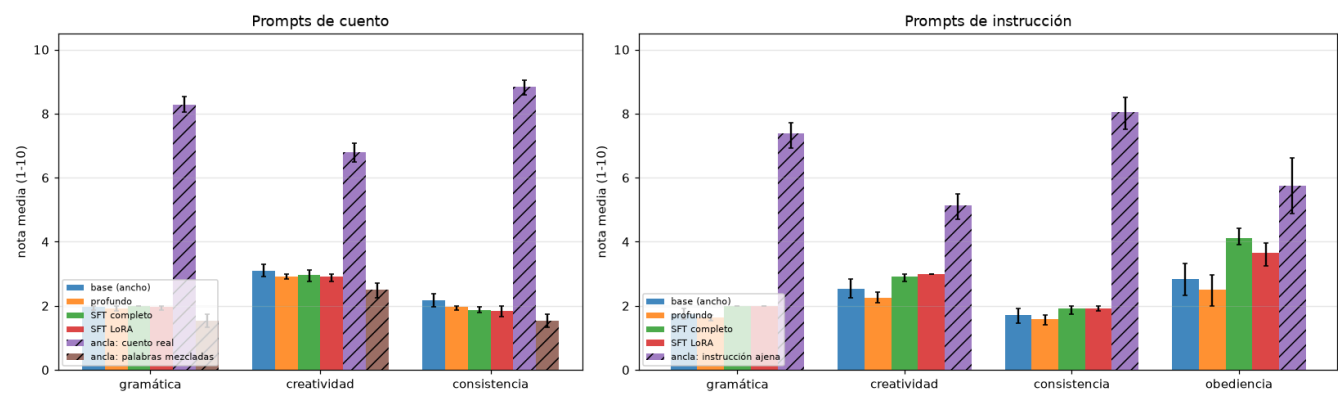


Figura 8. Notas del juez (qwen3:4b) por modelo y por ancla, con intervalos bootstrap al 95 % que remuestrean prompts.

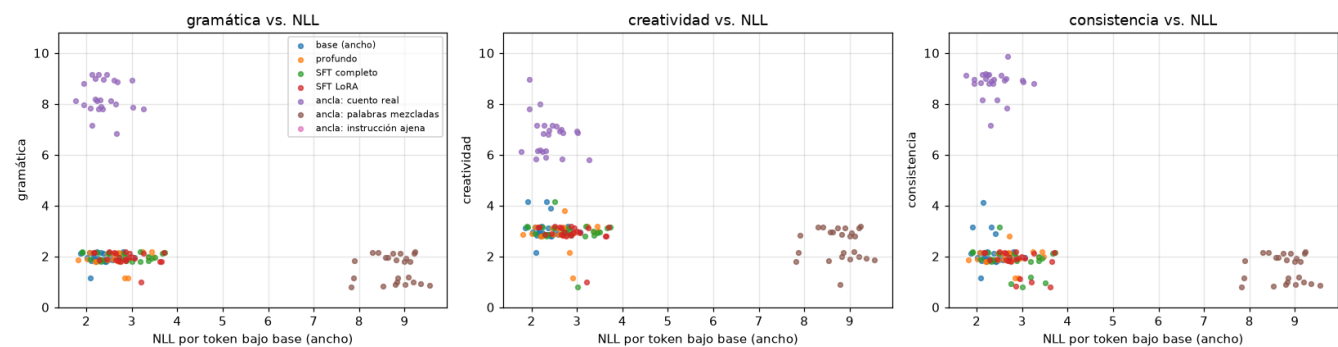


Figura 9. Nota del juez contra la NLL por token de cada texto, medida con el modelo ancho.

Corrida local (RTX 5070 Laptop, 8 GB), todas las etapas en orden, semilla 1337, juez **qwen3:4b** con `think=False` y `format="json"`. **264 evaluaciones**: 4 modelos × 48 textos (6 prompts de cuento y 6 de instrucción × 4 muestras) más 72 anclas — cuentos reales, esos mismos cuentos con las palabras mezcladas, y cuentos reales evaluados contra una instrucción que no les corresponde. Las cifras salen de `checkpoints/stage5/`. En la ejecución que quedó guardada en el notebook `05_judge` las respuestas del juez se leyeron de la caché de la primera corrida completa (`juez_respuestas.jsonl`), y por eso la celda del juez dice "listo en 0s"; esa primera vez, las 264 llamadas tardaron 165 segundos. Los textos evaluados son los mismos: las generaciones salen idénticas con la misma semilla.

1. ¿Es confiable el juez? Sí para gramática y coherencia, no para obediencia

- **Formato: 264 de 264 respuestas válidas** (`formato_juez.csv`), ninguna con texto extra ni con dos objetos. Con `format="json"`, la trampa de "el juez contesta dos veces" no apareció en esta corrida; el parser robusto quedó como seguro, sin llegar a usarse.
- **Separa bien las anclas** (`notas_por_modelo.csv`):

ancla	gramática	creatividad	consistencia
cuento real	8,3 [8,0, 8,5]	6,8 [6,5, 7,1]	8,8 [8,6, 9,0]
palabras mezcladas	1,5 [1,3, 1,8]	2,5 [2,2, 2,7]	1,5 [1,3, 1,8]

Mismo vocabulario y misma temática, con la gramática destruida: la nota cae de 8,3 a 1,5 y los intervalos no se pisan. El juez lee gramática y coherencia, no solo el tema.

- **La obediencia no es confiable.** Los cuentos reales evaluados contra una instrucción ajena (12,5 % de uso de las palabras pedidas) sacaron obediencia 5,75 [4,9, 6,6]. Los del SFT completo sacaron 4,0 tanto con 0 como con 3 de 3 palabras usadas. La correlación entre la obediencia del juez y el uso real de palabras es de apenas 0,23 (Spearman):

la obediencia está contaminada por la calidad general del texto (efecto halo). Para obediencia, la cuenta automática de la Etapa 4 es el mejor instrumento.

Notas por modelo (prompts de cuento; en obediencia y uso de palabras, prompts de instrucción):

	val ppl (liso)	gramática	creatividad	consistencia	obediencia	uso de palabras
base (ancho)	10,3	1,96	3,08	2,17 [2,0, 2,4]	2,8 [2,3, 3,3]	18 %
profundo	10,6	1,92	2,92	1,96 [1,9, 2,0]	2,5 [2,0, 3,0]	21 %
SFT completo	13,1	2,00	2,96	1,88	4,1 [3,9, 4,4]	56 %
SFT LoRA	12,7	1,96	2,92	1,83	3,7 [3,2, 4,0]	33 %

Hay un efecto piso. De las 192 generaciones de nuestros modelos, el juez puso gramática 2 en 173 y 1 en 19, nunca más. Contra la escala de un cuento real (8,3), los cuatro modelos están en el mismo escalón: el juez no puede ordenarlos en gramática. Las diferencias en consistencia son de décimas y, salvo base contra el resto, los intervalos se pisan.

2. Dónde le da la razón el juez a la perplejidad

- **Texto por texto, dentro de nuestros modelos:** cuanto menor la NLL de un texto (medida con el modelo ancho), mejor la nota. Spearman (`correlacion_nll_juez.csv`): gramática -0,35, creatividad -0,37, consistencia -0,44; con las anclas incluidas, -0,49, -0,49 y -0,52. El signo es el esperado en las tres, incluso en creatividad.
- **Base contra profundo:** la perplejidad prefiere al ancho (10,3 contra 10,6) y el juez también, por poco (consistencia 2,17 contra 1,96).
- **Contra la Etapa 2:** esa preferencia coincide con la probabilidad media de la respuesta correcta en los prompts de recuerdo (0,12 contra 0,10). No coincide con el reuso de palabras clave, que favorecería al profundo (0,58 contra 0,52). Ninguna de las tres lecturas reproduce "profundidad ↔ contexto" del paper a esta escala.

3. Dónde se le va para otro lado, y qué dice eso de la perplejidad

1. El SFT tiene la peor perplejidad y el juez no lo castiga.

- En texto liso, el SFT completo sube de 10,3 a 13,1 de perplejidad: es el peor de los cuatro.
- El juez no lo ve peor: gramática igual (2,0) y la obediencia más alta (4,1 contra 2,8).
- La perplejidad sobre TinyStories mide cuánto se parece el modelo a esa distribución. El SFT se corrió hacia otro formato, y eso se paga en perplejidad aunque los cuentos no empeoren.
- En su propia distribución (Instruct), el SFT baja de 3,20 a 2,12. La perplejidad depende de sobre qué texto se mide; la calidad no.

2. Un cuento real puntúa casi igual que el balbuceo del propio modelo.

- Con el modelo ancho como vara, los cuentos reales tienen una NLL de 2,39 por token, y las generaciones del propio ancho, 2,29: al modelo le parecen más probables sus propias muestras que los cuentos humanos.
- El juez las separa por más de 6 puntos (gramática 8,3 contra 1,96).
- La perplejidad bajo un modelo mide cuánto se parece un texto a lo que ese modelo produciría, no si es bueno. Por eso el propio modelo se autoevalúa bien.
- Solo el caso extremo de las palabras mezcladas (NLL 8,8) lo separan las dos varas por igual.

En una línea: la perplejidad sirve para detectar texto roto y para comparar modelos sobre la misma distribución; no sirve para decidir si un texto es un buen cuento, y penaliza cualquier cambio de formato como si fuera pérdida de calidad. El juez sirve justo para eso, con dos límites medidos: satura contra un piso cuando todos los modelos son malos, y su obediencia se deja llevar por la calidad general del texto.

4. Obediencia: el SFT sube, y acá la regex le gana al juez

- El SFT completo sube la obediencia del juez de 2,8 a 4,1 [3,9, 4,4] sobre los mismos prompts, y el uso de palabras de 18 % a 56 %; LoRA queda en el medio (3,7 y 33 %). En la dirección grande, las dos medidas coinciden.
- Texto por texto dejan de coincidir: Spearman 0,23. Los tres casos de mayor discrepancia son anclas de instrucción ajena — cuentos reales, bien escritos, con 0 de las palabras pedidas — a los que el juez puso obediencia 9, 7 y 7. Ahí la razón es la regex, no el juez: cuenta lo que dice contar, mientras que el juez mezcla obediencia con calidad.
- Por eso la conclusión de la Etapa 4 se apoya en uso – piso y no en la nota del juez.

Limitaciones

- 24 textos por modelo y tipo, y una sola semilla de entrenamiento por configuración. Alcanza para las diferencias grandes (anclas, obediencia del SFT), no para las de décimas.
- El juez tiene 4B parámetros. Con un juez más grande, el efecto piso probablemente se abriría en más escalones.

Cierre del proyecto: qué muestra el pipeline completo

La tesis de la consigna es "misma arquitectura, mejores datos → aparece el significado". Nuestra lectura, con nuestros números: aparece, pero en un orden. Primero la forma, después —y a medias— el contenido. Lo vimos tres veces, en tres etapas distintas, y coincide con lo que reporta el paper (la gramática llega antes que la consistencia):

1. **Preentrenamiento (Etapa 2).** El mismo GPT de la clase pasa de adivinar al azar (pérdida $9,06 \approx \ln 8.192$) a sus primeras oraciones bien formadas en unos 500 pasos (pérdida 3,3), sostiene un personaje con nombre desde el paso 2.000 (2,56) y termina en perplejidad 10,1. La coherencia del cuento entero, en cambio, no llega: el juez le pone consistencia 2,2 sobre 10, contra 8,8 de un cuento real.
2. **SFT (Etapa 4).** En 1.500 pasos (1,7 minutos) el modelo aprende el formato —pasa de 0 % a 92,5 % de cuentos cuando solo le damos la línea `Words: —`, mientras que usar las palabras pedidas se mueve mucho menos: la ganancia sobre el azar pasa de 0,11 a 0,32, y con la plantilla `solo Words` no cambia.
3. **Evaluación (Etapa 5).** La perplejidad mide forma: cuánto se parece un texto al corpus. Por eso castiga al SFT por cambiar de formato (10,3 → 13,1) aunque el juez no lo vea peor, y por eso el modelo encuentra más probables sus propias muestras (NLL 2,29) que los cuentos reales (2,39), que el juez separa por 6 puntos de gramática.

Cómo se encadenan las piezas. El tokenizador decide las unidades: con 8.192 entradas, casi cada palabra es un token (Etapa 1). El preentrenamiento les da relaciones: *dog* termina junto a *cat* y *puppy*, y *spoon* junto a *fork* y *bowl*, muy por encima de los controles al azar (Etapa 3); y además aprende la gramática local. El SFT no agrega conocimiento: reusa esas representaciones para instalar un comportamiento, y lo que más le cuesta es justo lo que pide contenido, como usar una palabra 50 a 100 tokens después de haberla leído. El juez, por último, muestra lo que la perplejidad no ve. Cada etapa se apoya en la anterior: los vecinos de la Etapa 3 existen porque el tokenizador hizo de *dog* una sola fila, y el SFT funciona tan rápido porque el base ya sabía escribir cuentos (con `Words + Story` arranca uno el 92,5 % de las veces antes de cualquier SFT).

Lo que no probamos, y habría que probar. Nuestro contraste con "datos peores" es el del paper y el del Transformer de caracteres sobre Shakespeare que vimos en clase, y ese contraste no mueve una sola variable: cambian los datos, pero también el tokenizador (caracteres contra BPE) y la escala. El experimento limpio es entrenar este mismo modelo, con los mismos pasos, sobre 82M tokens de texto web (lo planteamos en la Etapa 0). Con más tiempo, eso iría primero; después, tres semillas para la ablación y un SFT que enmascare la instrucción.

El registro de decisiones del proyecto —con qué contábamos, qué recortamos y por qué, y dónde terminamos contra lo planeado— sigue a continuación, junto con cómo usamos IA.

Registro de decisiones del proyecto

Las fechas salen del historial del repositorio y los tiempos, de los logs de las corridas.

Con qué contábamos al arrancar

- **Gente y tiempo:** cuatro integrantes y tres semanas de calendario (la consigna salió el 6/9 y se entrega el 27/9). En la práctica, el trabajo que quedó registrado se concentró en los últimos tres días, del 25/9 al 27/9. Ese fue el presupuesto real, y explica varias decisiones de abajo: todo tenía que correr en minutos, no en horas.
- **Máquinas:** Colab gratuito (T4 de 16 GB, sesiones que se cortan, disco que se borra), que es el entorno de la consigna; una notebook con RTX 5070 Laptop (8 GB de VRAM, 32 GB de RAM, Linux); y otra computadora con Windows.
- **Cuánto rendía una corrida:** lo medimos corriendo todo primero en modo prueba (`SMOKE_TEST`, unos 5 minutos) y después la corrida completa: 31 minutos en la RTX 5070 para las Etapas 0 y 2 a 5 (la 2, 17 minutos con los dos modelos; la 4, 8; la 5, 6). La Etapa 1 ya estaba corrida y tarda un par de minutos.

El plan: qué no se recortaba y qué sí

1. **Las Etapas 1 y 2, con la ablación, completas.** La 1 alimenta a todas, y la ablación es lo que convierte esto en un laboratorio.
2. **La Etapa 3 completa:** no entrena nada y corre en segundos en CPU, así que el esfuerzo podía ir a los controles de ruido.
3. **La Etapa 4 con el SFT completo** (sin él no hay Etapa 5); LoRA, solo si entraba.
4. **La Etapa 5 al final:** si faltaba tiempo, era lo primero en recortarse ("recorten desde el final").
5. **Tamaños elegidos para que la Etapa 2 entrara en minutos**, dentro de los rangos de la §7, y siempre una pasada de `SMOKE_TEST` antes de la corrida larga.

Diario

Fecha	Qué hicimos	Qué decidimos, y por qué
25/9	Etapas 0 a 2	<code>V = 8.192</code> : el techo del rango de la consigna, cerca de los ~10K del paper; el barrido mostró que más arriba ya casi no se acortan los cuentos (quedó la duda contra 4.096). <code>block_size = 256</code> , porque entra el 90 % de los cuentos contra el 60 % con 192. Para la ablación, el par de la §7 con cabezas de 32 dimensiones en los dos modelos, así solo cambian el ancho y la cantidad de capas; contamos los parámetros antes de entrenar (7,2 % de diferencia). 400.000 cuentos y 5.000 pasos: menos de una pasada por el corpus (0,93 épocas) y unos 15 minutos para los dos modelos.
26/9	Etapa 3	Pares fijados antes de mirar (<i>dog-cat</i> , <i>happy-sad</i> , <i>dog-spoon</i>), 10.000 pares al azar como control y el máximo de una búsqueda como segundo piso de ruido; una sola PCA ajustada sobre los dos estados, para que los paneles compartan ejes.
26/9	Etapa 4	Cada ejemplo de SFT alineado al principio de su registro (no ventanas al azar), para que el modelo nunca vea un cuento sin su instrucción; <code>lr 1e-4</code> y 1.500 pasos, según la §7; LoRA a mano, rango 8, sobre las proyecciones de atención. Medimos una trampa: un prompt que termina en espacio o salto de línea hace que el modelo escupa bytes sueltos, así que los prompts se cortan en <code>Story: .</code>
26/9	Etapa 5	Anclas para el juez (cuento real, palabras mezcladas, instrucción ajena), porque sin ellas no sabíamos si el juez discrimina; caché de respuestas, para que un corte no obligue a repetir llamadas.
26/9	Infraestructura	Huella SHA-1 del tokenizer en cada checkpoint: volver a correr la Etapa 1 pisaba el tokenizer y dejaba a los modelos leyendo ids con otro significado, sin ningún error. Un script corre las seis etapas en orden y guarda los notebooks ejecutados.
26/9	Corrida completa	Primero en modo prueba y después la completa, en la notebook con RTX 5070 en lugar de Colab (ver "Dónde terminamos").
27/9	Informe	Un informe por etapa dentro de su notebook, repartidos entre los cuatro; revisión final contra la consigna.

Qué se rompió o nos sorprendió, y qué hicimos

- **Un notebook perdió sus salidas** al regenerarlo desde su fuente `.py`: lo restauramos desde la copia ejecutada.
- **El chequeo de GPU podía colgarse** (drivers de GPU híbrida, o un proceso anterior de Python trabado con la GPU tomada): lo pasamos a un proceso aparte, con límite de tiempo.
- **`torch.load` se negaba a abrir un checkpoint** del SFT porque el historial guardaba escalares de numpy (desde torch 2.6 carga con `weights_only=True`): ahora se guardan como `float`.
- **La memoria de GPU de LoRA salía sesgada**: la medición incluía los modelos que ya estaban cargados (en la corrida de LoRA, también el de SFT completo). Ahora se mide por encima de lo residente.
- **Texto mal codificado en el corpus** (6 % de los cuentos), que encontramos recién al analizar la Etapa 1.
- **Resultados que había que leer con cuidado**: la cota inferior del intervalo del base en la Etapa 4 es un cero con error de redondeo ($3,5 \cdot 10^{-18}$), no una ganancia sobre el azar; y el juez, que separa muy bien las anclas, no sirve para medir obediencia (efecto halo, Etapa 5).

Qué recortamos, por qué, y qué nos costó

Recorte	Por qué	Qué nos costó
Una sola semilla por configuración	Cada semilla es otra corrida de la Etapa 2 y de todo lo que sigue; preferimos cubrir las cinco etapas	Las diferencias de décimas entre el ancho y el profundo no se pueden separar del ruido
400.000 cuentos y 5.000 pasos (0,93 épocas)	Que la Etapa 2 entrara en unos 15 minutos, casi sin repetir datos	Quedamos en el 70 % de la referencia de Chinchilla contando todos los parámetros, y los dos modelos seguían bajando al cortar
No comparar $V = 4.096$ contra 8.192	Exigía repetir la Etapa 2 entera	La duda de la Etapa 1 queda abierta
No limpiar el texto mal codificado	Lo vimos cuando el tokenizer ya lo usaban las demás etapas	Los modelos a veces generan <code>â€œ</code>
SFT sin enmascarar la instrucción ni filtrar a registros con <code>Words</code> :	Formato de continuación pelado, como pide la consigna, y un único presupuesto de 1.500 pasos	El contenido se movió a medias (Etapa 4)
Pocas muestras: 40 generaciones por celda en la Etapa 4 y 24 textos por modelo y tipo en la 5	Las fijamos antes de correr, sin estimar cuántas hacían falta. En retrospectiva no era un límite real: el juez evaluó los 264 textos en menos de 3 minutos	Solo se pueden leer las diferencias grandes; con más muestras se habrían podido resolver las de décimas
Sin un control con texto web	Otra corrida completa de las Etapas 1 y 2	La tesis "mejores datos" se apoya en el paper y en el modelo de Shakespeare de la clase, no en un control propio (ver el cierre del proyecto)

Dónde terminamos, contra el plan

- **Hicimos las cinco etapas**, incluidas las dos partes opcionales (LoRA y el juez). No hizo falta recortar etapas: el recorte fue en profundidad dentro de cada una (una semilla, subconjuntos, pocas muestras).
- **El cambio más grande fue dónde corrimos**. La consigna apunta a Colab, y los notebooks siguen preparados para eso (primera celda, bloque de Drive), pero la corrida final la hicimos en la notebook con RTX 5070: el pipeline entero tardaba poco más de media hora y así no dependíamos de sesiones que se cortan ni de montar Drive para no perder los checkpoints. El costo: no medimos cuánto tarda en una T4.
- **Lo que no esperábamos encontrar** terminó siendo de lo más interesante: que el SFT instala la forma mucho antes que el contenido, que LoRA casi no ahorra memoria a esta escala, que el juez separa las anclas pero no sabe medir obediencia, y que un corpus pensado como "limpio" trae un 6 % de texto mal codificado.

Uso de IA

Usamos un asistente de programación (Claude, de Anthropic), y queda a la vista en el historial del repositorio: los commits que escribió van firmados por él.

- **Código:** escribió la mayor parte del código de las Etapas 4 y 5 y de la infraestructura (el script que corre todas las etapas en orden, la huella del tokenizer, la medición de memoria) a partir de lo que le pedimos; nosotros lo corrimos, lo revisamos y decidimos qué medir.
- **Resultados y borradores:** ordenó en tablas los números medidos de cada etapa y redactó un primer borrador de los resultados de las Etapas 4 y 5.
- **Revisión final (27/9):** revisó los notebooks contra la consigna, chequeó los números citados contra las salidas y los CSV, corrigió afirmaciones que habían quedado desactualizadas y redactó, a partir de lo ya medido, la ampliación de la Etapa 0 (las respuestas cortas las escribimos nosotros), este registro y el cierre del proyecto. También armó el PDF con las conclusiones.

Dónde no le creímos. El resumen preliminar de resultados que armó el asistente comparaba el final de las curvas de los dos modelos de la ablación en tramos distintos, y atribuía al corpus, sin verificarlo, el $\hat{\alpha}$ que generó uno de ellos. En el informe de la Etapa 2 los comparamos en el mismo tramo (del paso 4.250 al 5.000: $-0,020$ el ancho contra $-0,025$ el profundo) y tratamos el $\hat{\alpha}$ como hipótesis, hasta que la Etapa 1 midió el texto mal codificado del corpus. También agregamos lo que el resumen no decía: que repetir un nombre no es recordarlo (en las muestras, Mia termina huyendo de Mia).

Anexo · Etapa 1: cómo medimos lo que no imprimen las celdas

Estos números no salen de las celdas del notebook 01_tokenizer: los medimos aparte, con el tokenizer.json guardado y los mismos cuentos. Para no tocar el código del notebook los dejamos como texto. Se pueden pegar en una celda nueva al final, porque usan variables que ya existen (tokenizer, vocab, whole_words, train_texts, eval_texts, ours_total, story_lengths, gpt2, cl100k, train_bpe y SWEEP_TRAIN_STORIES). Debajo de cada uno está lo que imprime. Los fragmentos C y G tardan un par de minutos; el resto, segundos.

A · El piso de tokens y de qué están hechos

```
import re

pre = tokenizer.pre_tokenizer # corta en palabras y signos ANTES del BPE: ningún merge cruza esos cortes
piso = np.mean([len(pre.pre_tokenize_str(s)) for s in eval_texts])
print(f"piso: {piso:.2f} tokens/cuento · BPE propio: {ours_total / len(eval_texts):.2f}")

palabras = [p for s in eval_texts[:2_000] for p, _ in pre.pre_tokenize_str(s) if re.search("[A-Za-z]", p)]
partidas = sum(len(tokenizer.model.tokenize(p)) > 1 for p in palabras)
print(f"palabras partidas en más de un token: {partidas / len(palabras):.1%}")

tokens = [t for e in tokenizer.encode_batch(eval_texts) for t in e.tokens]
print(f"tokens sin ninguna letra: {np.mean([not re.search('[A-Za-z]', t) for t in tokens]):.0%}")
saltos_cl100k = sum("\n" in cl100k.decode([i]) for ids in cl100k.encode_ordinary_batch(eval_texts) for i in ids)
print(f"tokens con salto de línea por cuento · BPE propio {sum('Ĉ' in t for t in tokens) / len(eval_texts):.1f} "
      f"· cl100k {saltos_cl100k / len(eval_texts):.1f}")
```

Imprime:

```
piso: 193.62 tokens/cuento · BPE propio: 195.75
palabras partidas en más de un token: 0.9%
tokens sin ninguna letra: 17%
tokens con salto de línea por cuento · BPE propio 7.2 · cl100k 3.7
```

B · En qué merge nace cada token, acá y en GPT-2

```
# nuestro: id 0 = <|endoftext|> e ids 1-256 = bytes → merge = id - 256 · GPT-2: ids 0-255 = bytes → merge = id - 255
for t in ["Ġthe", "ed", "Ġhe", "Ġwas", "Ġsaid", "Ġshe", "Ġof", "ic", "ion"]:
    print(f"{t!r:>8}: merge {tokenizer.token_to_id(t) - 256:>4} nuestro · "
          f"{gpt2.encode_single_token(t.replace('Ġ', ' ')) - 255:>4} en GPT-2")
print({t: tokenizer.token_to_id(t) - 256 for t in ["ĠLily", "Ġhappy", "Ġfriend", '."', '!"', "Once", "Ġonce", "Ġonce"]})
print(tokenizer.encode("Kitty smiled at Kitty").tokens, [gpt2.decode([i]) for i in gpt2.encode_ordinary("Kitty smiled at Kitty")])
```

Imprime:

```
'Ġthe': merge    7 nuestro ·    7 en GPT-2
'ed': merge     8 nuestro ·   21 en GPT-2
'Ġhe': merge    23 nuestro ·   84 en GPT-2
'Ġwas': merge   24 nuestro ·  118 en GPT-2
'Ġsaid': merge  72 nuestro ·  276 en GPT-2
'Ġshe': merge   79 nuestro ·  418 en GPT-2
'Ġof': merge    88 nuestro ·   31 en GPT-2
'ic': merge    214 nuestro ·   36 en GPT-2
'ion': merge   503 nuestro ·   40 en GPT-2
{'ĠLily': 108, 'Ġhappy': 146, 'Ġfriend': 158, '."', '!"', 'Once': 169, 'Ġonce': 2650, 'Ġonce': 5344}
['Kitty', 'Ġsmiled', 'Ġat', 'ĠKitty'] ['K', 'itty', ' smiled', ' at', ' Kitty']
```

C · De qué está hecha la "U" del barrido, y cuánto texto cubren los tokens más usados

```
def contar_uso(tok, textos, chunk=10_000):
    """Cuántas veces aparece cada id del vocabulario al tokenizar `textos` (de a `chunk` cuentos, por memoria)."""
    uso = np.zeros(tok.get_vocab_size(), dtype=np.int64)
    for i in range(0, len(textos), chunk):
        ids = np.concatenate([np.asarray(e.ids, dtype=np.int64) for e in tok.encode_batch(textos[i : i + chunk])])
        uso += np.bincount(ids, minlength=len(uso))
    return uso

# "intermedios": no aparecen ni en los cuentos con los que se entrenó ese tokenizador (un merge posterior se los comió)
# "raros": aparecen en esos cuentos, pero no en los 5.000 held-out
for vocab_size in (512, 4_096, 8_192, 16_384):
    cand = train_bpe(train_texts[:SWEEP_TRAIN_STORIES], vocab_size)
    uso_val, uso_train = contar_uso(cand, eval_texts), contar_uso(cand, train_texts[:SWEEP_TRAIN_STORIES])
    ids = np.arange(vocab_size)
    sin_usar = uso_val == 0
    print(f"V={vocab_size}>6,: sin usar {sin_usar.sum():>5,} = <|endoftext|> {int(sin_usar[0])} "
          f"+ bytes {(sin_usar & (ids >= 1) & (ids <= 256)).sum()} "
          f"+ merges intermedios {(sin_usar & (ids > 256) & (uso_train == 0)).sum()} "
          f"+ merges raros {(sin_usar & (ids > 256) & (uso_train > 0)).sum()}")

print({c: sum(s.count(c) for s in eval_texts) for c in "(){}[]=<>#"} # símbolos de código en los 5.000 cuentos
      uso_final = contar_uso(tokenizer, eval_texts)
      print(f"tokenizador final, sin usar en el held-out: {(uso_final == 0).mean():.2%} · "
            f"los 1.000 tokens más usados cubren el {np.sort(uso_final)[::-1][1_000].sum() / uso_final.sum():.1%} del texto")
```

Imprime:

```
V= 512: sin usar 162 = <|endoftext|> 1 + bytes 161 + merges intermedios 0 + merges raros 0
V= 4,096: sin usar 372 = <|endoftext|> 1 + bytes 167 + merges intermedios 75 + merges raros 129
V= 8,192: sin usar 1,348 = <|endoftext|> 1 + bytes 170 + merges intermedios 401 + merges raros 776
V=16,384: sin usar 7,547 = <|endoftext|> 1 + bytes 183 + merges intermedios 2057 + merges raros 5306
{'(': 0, ')': 0, '[': 0, ']': 0, '{': 0, '}': 0, '=': 0, '_': 0, '<': 0, '>': 0, '#': 0}
tokenizador final, sin usar en el held-out: 15.55% · los 1.000 tokens más usados cubren el 90.2% del texto
```

D · Cuántas ventanas al azar contienen un cuento entero

```
# Si el entrenamiento corta ventanas de B tokens en posiciones al azar del corpus concatenado, una ventana contiene
# entero un cuento de largo L solo si arranca en una de las B - L + 1 posiciones que lo dejan adentro.
for block_size in (128, 192, 256):
    ventanas = np.maximum(0, block_size + 1 - story_lengths).sum() / story_lengths.sum()
    no_entran = story_lengths[story_lengths > block_size].sum() / story_lengths.sum()
    print(f"block_size {block_size}: ~{ventanas:.0%} de las ventanas contiene un cuento entero · "
          f"{no_entran:.0%} de los tokens son de cuentos que no entran")
```

Imprime:

```
block_size 128: ~1% de las ventanas contiene un cuento entero · 97% de los tokens son de cuentos que no entran
block_size 192: ~10% de las ventanas contiene un cuento entero · 51% de los tokens son de cuentos que no entran
block_size 256: ~36% de las ventanas contiene un cuento entero · 18% de los tokens son de cuentos que no entran
```

E · Texto mal codificado, y cuántos cuentos arrancan igual

```
con_mojibake = sum("â€" in s for s in eval_texts)
print(f"cuentos con 'â€': {con_mojibake} ({con_mojibake / len(eval_texts):.1%}")
print(tokenizer.encode("I donâ€™t know").tokens, "vs", tokenizer.encode("I don't know").tokens)
```

```
print(f"arrancan con 'Once upon a time': {sum(s.startswith('Once upon a time') for s in eval_texts):,} de {len(eval_texts):,}")
```

Imprime:

```
cuentos con 'â€': 302 (6.0%)
['I', 'Ġdon', 'âĤ', 't', 'Ġknow'] vs ['I', 'Ġdon', "'t", 'Ġknow']
arrancan con 'Once upon a time': 3,327 de 5,000
```

F · SolidGoldMagikarp

```
print(gpt2.encode_single_token(" SolidGoldMagikarp")) # es un solo token del vocabulario de GPT-2
```

Imprime:

```
43453
```

G · Cuántos "tokens de palabra entera" son palabras de verdad

```
# Un token cuenta como palabra completa cuando él solo ocupa todo un pre-token (una palabra entre espacios o signos).
n_total = np.zeros(len(vocab), dtype=np.int64)
n_solo = np.zeros(len(vocab), dtype=np.int64)
for i in range(0, len(train_texts), 10_000):
    for e in tokenizer.encode_batch(train_texts[i : i + 10_000]):
        if not e.ids:
            continue # hay algunos cuentos vacíos
        ids = np.asarray(e.ids)
        w = np.asarray([-1 if x is None else x for x in e.word_ids]) # a qué pre-token pertenece cada token
        solo = np.r_[True, w[1:] != w[:-1]] & np.r_[w[1:] != w[:-1], True]
        np.add.at(n_total, ids, 1)
        np.add.at(n_solo, ids[solo], 1)

candidatos = np.array([vocab[t] for t in whole_words])
vistos = n_total[candidatos] > 0
de_verdad = candidatos[vistos & (n_solo[candidatos] >= n_total[candidatos] / 2)]
comienzos = candidatos[vistos & (n_solo[candidatos] < n_total[candidatos] / 2)]
print(f"casí siempre palabra completa: {len(de_verdad):,} ({len(de_verdad) / len(vocab):.0%} del vocabulario) · "
      f"casí siempre comienzo de palabra: {len(comienzos):,} · nunca aparecen: {(~vistos).sum():,}")
print({t: (int(n_total[vocab[t]]), int(n_solo[vocab[t]])) for t in ["ĠBu", "Ġru", "Ġequ", "Ġday"]})
print("comienzos:", sorted((t for t in whole_words if n_total[vocab[t]] > 100 and n_solo[vocab[t]] == 0),
key=vocab.get)[:10])
print("nunca vistos:", sorted((t for t in whole_words if n_total[vocab[t]] == 0), key=vocab.get)[:10])
```

Imprime:

```
casí siempre palabra completa: 4,917 (60% del vocabulario) · casí siempre comienzo de palabra: 949 · nunca aparecen: 100
{'ĠBu': (86, 0), 'Ġru': (126, 0), 'Ġequ': (45, 0), 'Ġday': (126877, 126815)}
comienzos: ['Ġh', 'Ġwa', 'Ġg', 'Ġpl', 'Ġwh', 'Ġne', 'Ġsm', 'Ġbo', 'Ġse', 'Ġj']
nunca vistos: ['Ġbeaut', 'Ġsurpr', 'Ġadvent', 'Ġdiffere', 'Ġfav', 'Ġpare', 'Ġunderst', 'Ġbutterf', 'Ġgi', 'Ġdro']
```